



DESIGN A RELIABLE PDF SEARCH AGENT

Software-Entwicklungspraktikum (SEP)

Sommersemester 2026

Technical Design

Customer

Technische Universität Braunschweig

Institute of Artificial Intelligence

Prof. Dr. Michel Besserve

Mühlenpfordtstraße 23

38106 Braunschweig

Supervisor: Baptiste Brémond

Contractor:

Name	E-Mail Address
Wajd Abdallah	w.abdallah@tu-raunschweig.de
Raghdaa Khoder	r.khoder@tu-braunschweig.de
Yosr Kharrat	y.kharrat@tu-braunschweig.de
Mouhib Hchaichi	m.hchaichi@tu-braunschweig.de

Braunschweig, 22. Juni 2026

Table of Contents

Chapter	Authors	Comments
2	Wajd Abdallah	Checked by RaghdAA Khoder, Mouhib Hchaichi, Yosr Kharrat
3	RaghdAA Khoder	Checked by Wajd Abdallah, Mouhib Hchaichi, Yosr Kharrat
4	RaghdAA Khoder	Checked by Mouhib Hchaichi, Yosr Kharrat
5	Mouhib Hchaichi	Checked by RaghdAA Khoder, Yosr Kharrat, Wajd Abdallah
6	Wajd Abdallah	Checked by RaghdAA Khoder, Mouhib Hchaichi, Yosr Kharrat
7	Mouhib Hchaichi	Checked by RaghdAA Khoder, Yosr Kharrat
8	Yosr Kharrat	Checked by RaghdAA Khoder, Mouhib Hchaichi, Wajd Abdallah
9	Yosr Kharrat	Checked by RaghdAA Khoder, Mouhib Hchaichi, Wajd Abdallah
10	Yosr Kharrat	Checked by RaghdAA Khoder, Mouhib Hchaichi, Wajd Abdallah
11	Wajd Abdallah, Yosr Kharrat, RaghdAA Khoder, Mouhib Hchaichi	Checked by all team members

Table of Contents

1	AI Statement	7
	AI Statement	7
2	Introduction	8
2.1	Project Overview	8
2.2	Project Details	11
2.2.1	PDF Processing and Indexing	11
2.2.2	Question Answering	12
2.2.3	Evaluation Pipeline	13
3	Analysis of Product Functions	18
3.1	Analysis of Function	18
4	Resulting Software Architecture	31
4.1	Component Specification	31
4.2	Interface Specification	34
4.2.1	Interfaces	34
4.3	Protocols for the Use of the Components	36
5	Verteilungsentwurf	40
6	Implementation Design	42
6.1	Implementation of Component ⟨C10⟩: User Interface	42
6.1.1	Class Diagram	43
6.1.2	Description	44
6.2	Implementation of Component ⟨C20⟩: Document Ingestion	45
6.2.1	Class Diagram	46

6.2.2	Description	46
6.3	Implementation of Component $\langle C30 \rangle$: Retrieval Engine	49
6.3.1	Description	50
6.4	Implementation of Component $\langle C40 \rangle$: Evaluation Module	52
6.4.1	Description	52
6.5	Implementation of Component $\langle C50 \rangle$: Embedding Service	54
6.6	Implementation of Component $\langle C60 \rangle$: Data Storage	54
7	Data Model	57
7.1	Diagramm	57
7.2	Explanation	59
8	Configuration	63
8.1	Runtime Configuration	63
8.2	Document Storage Configuration	64
8.3	Retrieval Configuration	65
8.4	Language Model Configuration	65
8.5	User Interface Configuration	66
8.6	Evaluation Configuration	67
8.7	Summary	67
9	Changes Compared to the Functional Design	69
9.1	Architecture Refinement	69
9.2	PDF Processing Pipeline	70
9.3	Retrieval Pipeline	70
9.4	Answer Generation	71
9.5	Abstention Mechanism	71
9.6	Data Model	72
9.7	Configuration	72
9.8	Evaluation	73
9.9	Summary	73

10 Fulfilment of Requirements	74
10.1 Mandatory Requirements	74
10.2 Desired Requirements	76
10.3 Optional Requirements	77
11 Glossary	79

List of Figures

2.1	Statechart: System Overview of the PDF Search Agent	10
2.2	Activity Diagram: PDF Processing and Indexing	15
2.3	Activity Diagram: Question Answering	16
2.4	Activity Diagram: Evaluation Pipeline	17
3.1	Sequenzdiagramm F10	25
3.2	Sequenzdiagramm F20	26
3.3	Sequenzdiagramm F30	27
3.4	Sequenzdiagramm F40	28
3.5	Sequenzdiagramm F50	29
3.6	Sequenzdiagramm F60	30
4.1	Component Diagram	33
4.2	Protocol Statechart: Embedding Service ⟨C50⟩	38
4.3	Protocol Statechart: Data Storage ⟨C60⟩	39
5.1	Deployment Diagram of the PDF Search Agent	41
6.1	Class Diagram: User Interface ⟨C10⟩	43
6.2	Class Diagram: Document Ingestion ⟨C20⟩	56
7.1	Data model of the PDF Search Agent	58

1 AI Statement

The AI tools used in this project are classified as **Type 0 –Assistive Support (Non-Generative)** according to the ISF AI Guidelines. This means that AI was only used for basic language support, such as spelling correction, grammar checking, and minor phrasing improvements, without adding new content, ideas, or technical decisions.

According to the ISF AI Guidelines, Type 0 usage does not require documentation or marking.

Braunschweig, 22. Juni 2026

Wajd Abdallah, Raghdah Khoder, Yosr Kharrat, Mouhib Hchaichi

2 Introduction

This chapter describes the overall design and behavior of the PDF Search Agent. It is carried over from the Functional Design and refined with technical annotations. It covers how the system works, which components are involved, and how the main workflows are structured. The system overview is presented using a Statechart that distinguishes between the two user roles. The three main workflows are described in detail using Activity Diagrams. The following chapters cover the system architecture, the product functions, the data model, the implementation design of each component, the user interface, and the technical environment.

2.1 Project Overview

The goal of this project is to build a system that allows users to upload PDF documents, ask questions in natural language, and receive answers that are grounded in the content of the uploaded documents. Every answer includes a citation showing the source document name and page number so that the user can verify the information directly.

The core problem this project addresses is that generic AI systems may answer questions based on prior training rather than on the uploaded documents. As a result, answers cannot be verified and may contain invented information, a problem known as **hallucination**.

The PDF Search Agent addresses this by using **Retrieval-Augmented Generation (RAG)**. Relevant text passages are first retrieved from the uploaded documents using semantic similarity search and then used as context for generating a grounded answer via a local language model. If no sufficient evidence is found, the system returns an **abstention message** instead of producing an unsupported response. This abstention mechanism is implemented in `generator.py` and is triggered either when the cosine similarity score of the retrieved chunks falls below a configurable threshold, or when the language model response contains the phrase *could not find*.

The second central part of the project is the **evaluation pipeline**. The system is not only a working prototype with a user interface, but a measurable and reproducible solution whose quality can be assessed systematically. The evaluation pipeline is implemented in the `evaluation/` directory and runs the full RAG pipeline on the **Open RAG Benchmark**, a standardized dataset of 1,000 arXiv PDFs and 3,045 QA pairs. Performance is measured using the **RAGAS** evaluation framework, which computes metrics including faithfulness, answer relevancy, context recall, and context precision automatically. In addition, the pipeline computes abstention rate, citation correctness, and average latency independently. These metrics allow the project team to justify design decisions such as chunk size, retrieval parameters, and model selection with concrete and reproducible numbers.

The full system is implemented in Python and consists of the following modules: `parser.py` for PDF validation and text extraction, `chunker.py` for splitting extracted text into segments, `indexer.py` for embedding generation and vector storage using ChromaDB, `retriever.py` for semantic search, `generator.py` for answer generation and abstention handling, `pipeline.py` for orchestrating the full RAG workflow, and the `evaluation/` package for automated quality measurement. The user interface is built with Streamlit and provides two views: a search interface for standard users and an administration and evaluation dashboard for administrators.

The Statechart in Figure 2.1 shows the overall behavior of the system, divided by user role. The standard user interacts with the system through the main search interface to upload documents and ask questions. The administrator can additionally access the evaluation and administration dashboard to monitor system behavior, run evaluation pipelines, and inspect quality metrics.

The Statechart shows the following states and transitions. The system begins in the state **Main Interface open** as soon as the user opens the application in a browser. From there, the system transitions to **Waiting for PDF upload**, where it remains until the user selects a file. If the user uploads an invalid file, for example a non-PDF or an empty file, the system transitions to **Showing error message** and returns immediately to **Waiting for PDF upload**, allowing the user to try again. The error is raised in `parser.py` and displayed in the Streamlit sidebar.

If the user uploads a valid PDF, the system transitions to **Indexing in progress**. During this state, `parser.py` extracts the text content page by page, `chunker.py` splits the content into segments, `indexer.py` converts each segment into a 384-dimensional embedding vector

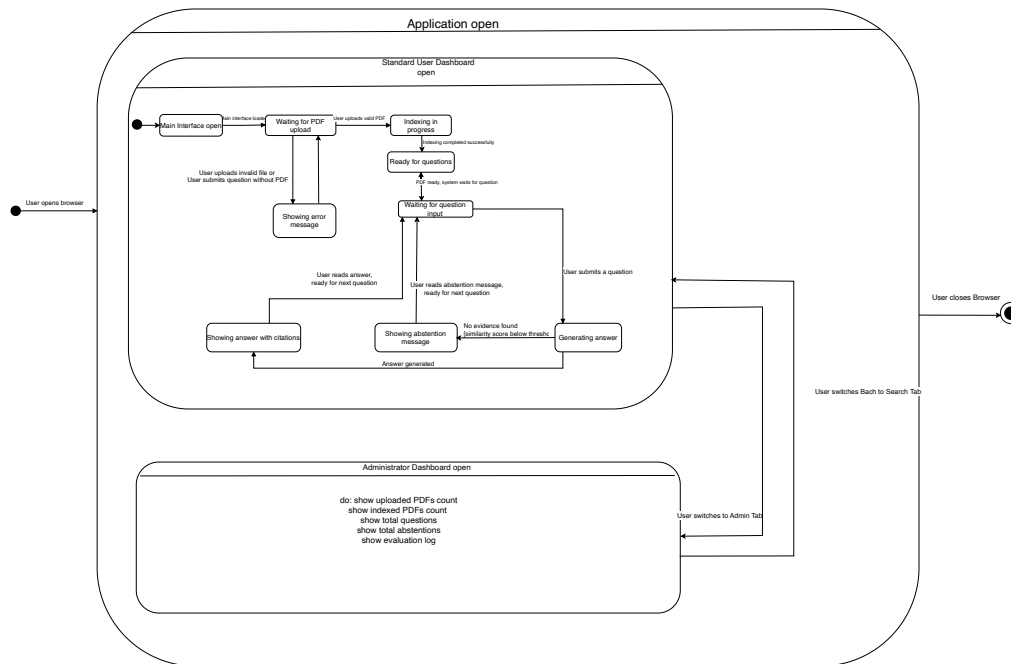


Figure 2.1: Statechart: System Overview of the PDF Search Agent

using the Sentence Transformers model **all-MiniLM-L6-v2**, and stores the vectors in the local ChromaDB vector database. Once this process completes successfully, the system transitions to **Ready for questions** and then immediately to **Waiting for question input**.

From **Waiting for question input**, two paths are possible. If the user submits a question without having indexed a document, or submits an empty question, the system transitions to **Showing error message** and returns to **Waiting for question input**. These checks are performed in `pipeline.py`. If a valid question is submitted with an indexed document available, the system transitions to **Generating answer**.

From **Generating answer**, `retriever.py` performs a cosine similarity search in ChromaDB and passes the top-k chunks to `generator.py`. The system either transitions to **Showing answer with citations** if sufficient evidence was found, or to **Showing abstention message** if the similarity score is below the configured threshold. In both cases, after the user reads the response, the system returns to **Waiting for question input**.

The administrator can switch to the **Admin Dashboard open** state at any time by clicking the administration tab. The admin dashboard displays the total number of uploaded and indexed

documents, the total number of questions submitted, the total number of abstentions, and a structured evaluation log. The administrator can return to the main interface at any time by switching back to the Search tab.

The entire system remains in the outer state **Application open** for as long as the browser session is active. When the user closes the browser, the system transitions to the final end state.

2.2 Project Details

The following subsections describe the three main workflows of the system in technical detail. Each workflow is described using an Activity Diagram with swimlanes that show which module or component is responsible for each step.

2.2.1 PDF Processing and Indexing

After a user uploads a PDF document, the system processes it through three steps: parsing, chunking, and indexing. This workflow is shown in Figure 2.2.

The workflow begins in the **User** swimlane. The user opens the application in a browser, navigates to the upload area in the sidebar, selects a PDF file, and clicks the upload button.

Control then passes to the **Parser** swimlane, implemented in `parser.py` using the PyMuPDF library (`fitz`). The parser receives the file and checks whether it is a valid PDF by verifying the file extension, confirming the file is not empty, and checking that the file header begins with the standard PDF marker `%PDF`. If the file is invalid, a `ValueError` is raised and an error message is displayed in the Streamlit sidebar. Control returns to the user who can try again. If the file is valid, the parser reads the file metadata, including file name, file size, and upload timestamp, sets the indexing status to *Indexing...*, and extracts the text content page by page. Each page stores the page number and the extracted text. If no readable text can be extracted, for example because the document is a scanned image without OCR, an error message is shown and processing stops. If text extraction is successful, control passes to the **Chunker** swimlane.

The Chunker, implemented in `chunker.py`, splits the extracted text into smaller segments using a sliding window approach. The chunk size and chunk overlap are defined as configuration parameters and directly influence retrieval quality. Each chunk is stored as a Python dictionary containing the text content, the source document name, the page number, the chunk index within the document, and the token count. Control then passes to the **Indexer** swimlane.

The Indexer, implemented in `indexer.py` using the Sentence Transformers library, converts each chunk into a 384-dimensional embedding vector using the model `all-MiniLM-L6-v2`. Each embedding vector together with its chunk metadata is stored in the local ChromaDB vector database. Once all chunks are stored, the Indexer sets the indexing status to *Indexed* and returns control to the User swimlane. The user now sees the file name and the status *Indexed* in the sidebar and can begin asking questions.

2.2.2 Question Answering

When a user submits a question, the system retrieves relevant document chunks and generates a grounded answer. This workflow is shown in Figure 2.3.

The workflow begins in the **User** swimlane. The user types a question into the input field at the bottom of the chat interface and clicks the Send button.

Control passes to the **Retriever** swimlane, implemented in `retriever.py`. The retriever first checks whether a PDF document has been indexed in the ChromaDB vector database. If no indexed document is available, an error message is returned and control goes back to the user. The retriever also checks whether the submitted question is empty and returns a corresponding error message if so. If the question is valid and a document is indexed, the retriever converts the question into a 384-dimensional embedding vector using the same Sentence Transformers model `all-MiniLM-L6-v2` used during indexing. It then queries ChromaDB using cosine similarity and returns the top-k most relevant chunks, where k is defined in the system configuration. Each returned chunk contains the text content, the source document name, the page number, and the similarity score.

Control then passes to the **Generator** swimlane, implemented in `generator.py`. The generator first checks whether the highest similarity score among the retrieved chunks is above the configured abstention threshold. If the score is below the threshold, the generator sets **abstained**

`= True`, `citations = []` and returns an abstention message directly to the user without calling the language model. If sufficient evidence is found, the generator builds a prompt using a predefined template that includes the user question and the retrieved chunks as labeled context passages with source identifiers. The prompt is sent to the Llama 3.2 model running via a local Ollama instance using the `ollama` Python library. The language model generates a response. The generator then checks whether the response contains the abstention phrase *could not find*. If so, `abstained = True` is set and an abstention message is returned. If the answer is valid, the generator extracts citations from the retrieved chunks, removes duplicates by source and page, and returns the final answer object containing the answer text, the list of citations, and `abstained = False`.

Control returns to the **User** swimlane. The user sees the generated answer displayed in the chat interface together with citation badges showing the source document name and page number. The user can then ask another question and the system returns to the waiting state.

2.2.3 Evaluation Pipeline

The evaluation pipeline is the second central component of this project alongside the RAG system itself. Its purpose is to measure and prove the quality of the system in a reproducible and standardized way. This workflow is shown in Figure 2.4.

The workflow begins in the **Administrator** swimlane. The administrator starts the evaluation pipeline by running `run_evaluation.py`.

Control passes to the **Benchmark Loader** swimlane, implemented in `benchmark_loader.py`. The Benchmark Loader loads a defined subset of questions and expected answers from the Open RAG Benchmark dataset. If the dataset is not available locally, an error message is shown and the evaluation stops. If the dataset is available, the Benchmark Loader prepares each QA pair, containing the question, the expected answer, the source document name, and the expected source page number, and passes them one by one to the **RAG Pipeline** swimlane.

For each question, the full RAG pipeline is executed by calling `pipeline.answerQuestion()`. The retriever converts the question into an embedding vector, searches ChromaDB for the most similar chunks, and returns the top-k results with similarity scores. The generator then checks whether the similarity score is above the abstention threshold. If not, `abstained = True` is

set and an abstention result is stored. If the score is sufficient, the generator builds a prompt, sends it to the LLM via the Ollama instance, and checks whether the response contains an abstention phrase. If yes, `abstained = True` is set. If no, the generator extracts citations and stores the full result as a structured dictionary containing the question, the retrieved chunks with similarity scores, the generated answer, the citations, the abstention flag, and the response time in milliseconds. After all questions have been processed, all stored results are passed to the **RAGAS** swimlane.

RAGAS, via `eval_ragas.py`, computes the following metrics automatically: **Faithfulness** measures whether the generated answer is supported by the retrieved chunks; **Answer Relevancy** measures whether the answer addresses the question; **Context Recall** measures whether the retrieved chunks contain all needed information; and **Context Precision** measures whether the retrieved chunks are relevant and free of noise. In addition, `metrics.py` computes **Abstention Rate**, **Citation Correctness**, and **Average Latency** independently. All scores are saved to `evaluation/results/scores.json` and `scores.csv` together with the configuration used for that run, making every evaluation run fully reproducible. The results are returned to the Administrator, who can inspect them in the evaluation dashboard.

DESIGN A RELIABLE PDF SEARCH AGENT

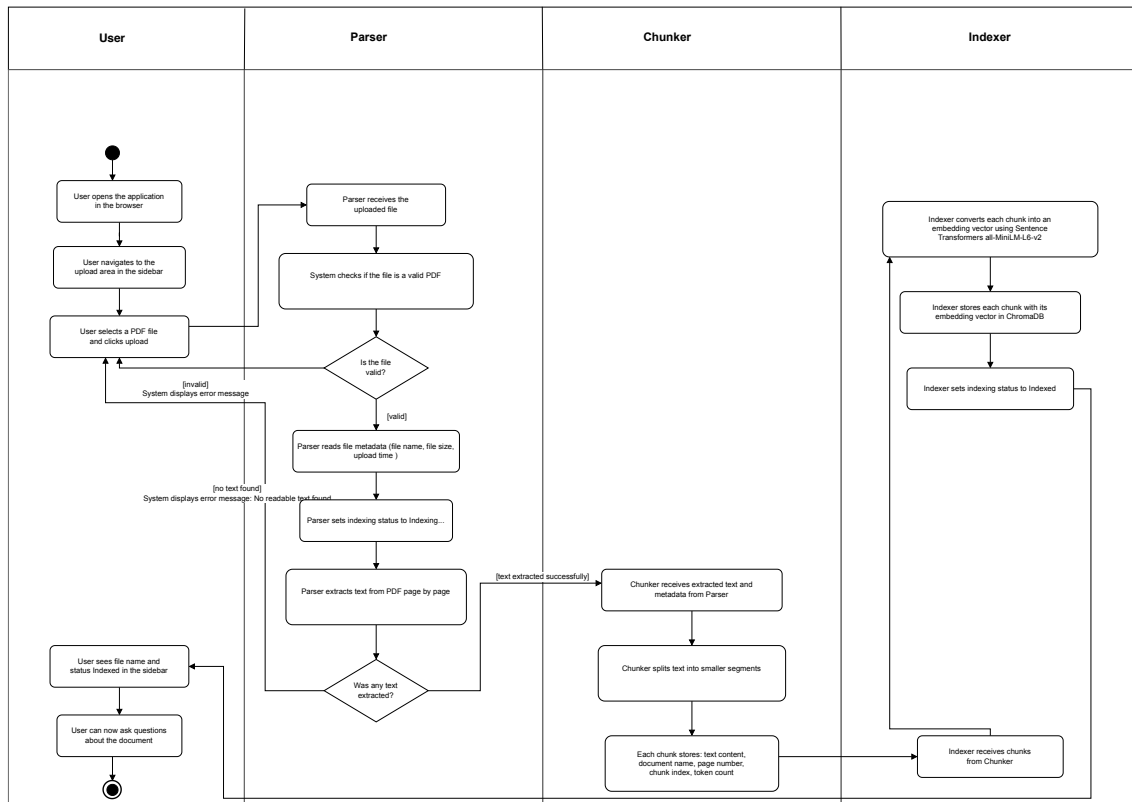


Figure 2.2: Activity Diagram: PDF Processing and Indexing

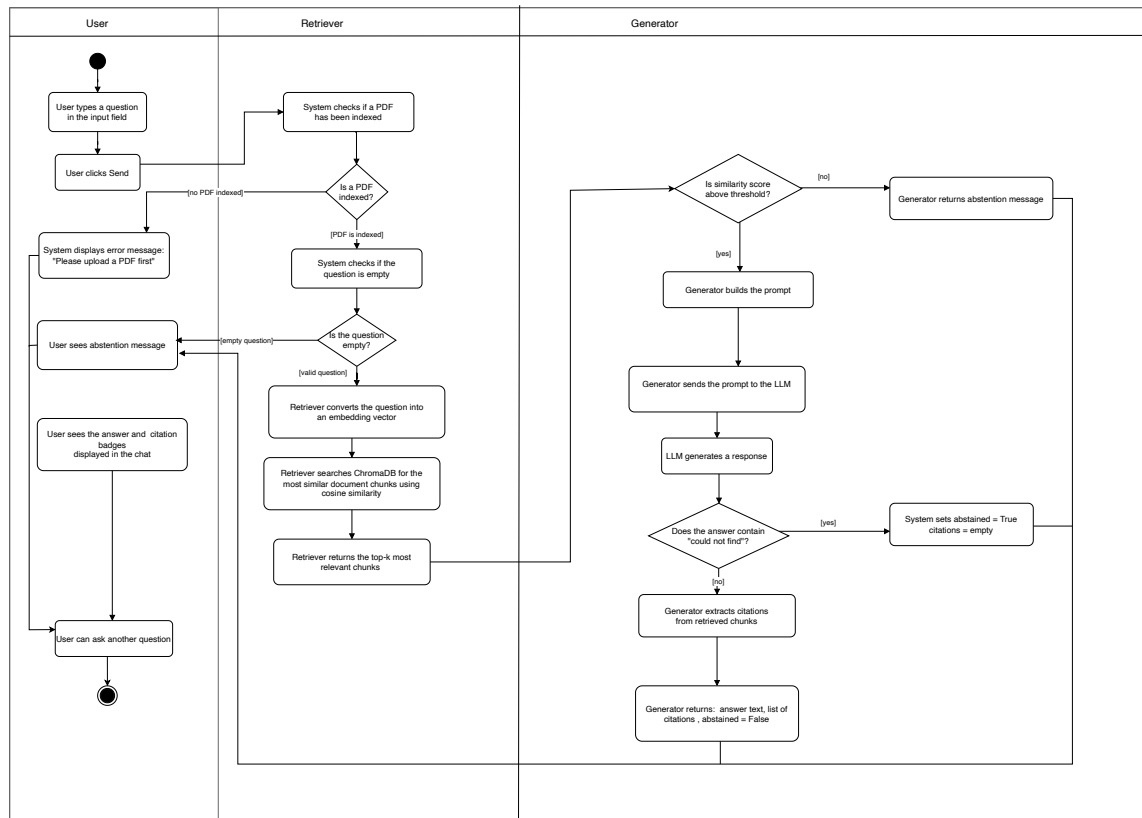


Figure 2.3: Activity Diagram: Question Answering

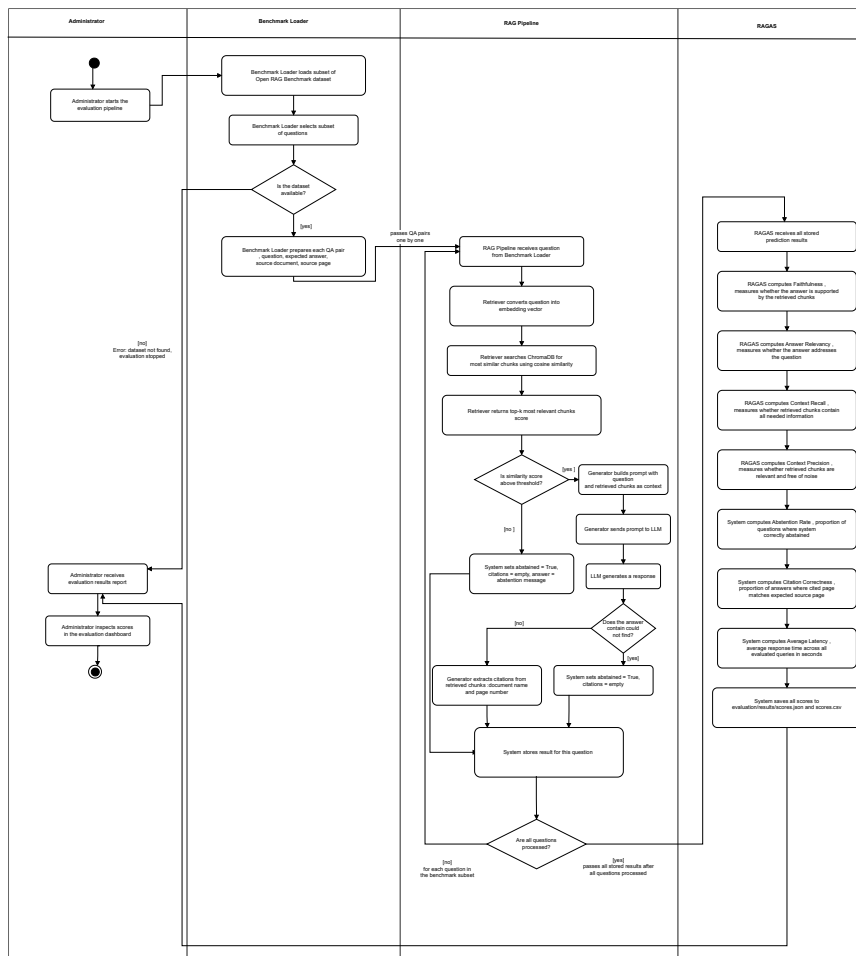


Figure 2.4: Activity Diagram: Evaluation Pipeline

3 Analysis of Product Functions

A thorough understanding of the system's functionality is essential for designing a robust and scalable architecture. The following sections examine each product function in detail, focusing on the interactions between system components and the processes required to fulfill the specified requirements.

Each function is analyzed individually to provide a clear view of its behavior, responsibilities, and dependencies within the system. In addition, relevant non-functional requirements are considered wherever applicable to ensure that quality attributes such as reliability, performance, and usability are adequately addressed.

Separate subsections are provided for all product functions defined in the requirements specification, allowing a structured and comprehensive analysis of the system.

3.1 Analysis of Function

Analysis of Function F10: PDF Import and Validation (RM1, RM4):

Use Case: Upload and validate a PDF document. Actors: User, System Trigger: The user selects a file and initiates the upload via the interface. Preconditions:

The system is running and the upload interface is accessible.

Description:

1. The user uploads a file via the document upload interface.
2. The system checks whether the file has a valid PDF format.
3. The system verifies that the file content is readable and not corrupted.

4. If valid, the file is accepted and passed to F20 for further processing.
5. If invalid or unreadable, the system rejects the file and displays a clear error message without crashing.

Postconditions (Success): The file is accepted and its name and status are displayed in the interface. Processing continues in F20.

Postconditions (Failure): An error message is displayed. The system remains operational.

Analysis of Function F20: PDF Content Extraction and Chunking (RM1):

Use Case: Extract and preprocess text content from a validated PDF.

Actors: System

Trigger: A valid PDF file has been successfully accepted by F10. Preconditions:

A valid, readable PDF document has been accepted by F10.

Description:

1. The system extracts the raw text from the PDF document page by page.
2. The extracted text is cleaned if necessary (e.g. removal of special characters or formatting artifacts).
3. The cleaned text is split into smaller, overlapping chunks of defined size.
4. Each chunk is stored together with its metadata (document name, page number) for further processing in F30.

Postconditions (Success): All text chunks with metadata are stored and ready for embedding generation in F30.

Postconditions (Failure): An error message is displayed. The system remains stable and does not crash.

Analysis of Function F30 Embedding Generation and Indexing (RM1):

Use Case: Generate semantic embeddings for text chunks and store them in the vector database.

Actors: System

Trigger: Text chunks have been successfully created and stored by F20. Preconditions:

Text chunks with metadata are available from F20. The embedding model is available and operational.

Description:

1. The system passes each text chunk to the embedding model.
2. The embedding model generates a numerical vector representation for each chunk.
3. The vectors are stored in the vector database together with their metadata (document name, page number, chunk ID).
4. The indexing status is updated and displayed in the interface.

Postconditions (Success): All embeddings are stored in the vector database and the document is ready for semantic search in F40.

Postconditions (Failure): An error message is displayed. The system remains stable and does not crash.

Analysis of Function F40: Query Processing and Retrieval (RM2):

Use Case: Process a natural-language query and retrieve the most relevant document chunks.

Actors: User, System

Trigger: The user submits a natural-language question via the search interface. Preconditions:

At least one document has been successfully indexed by F30. The embedding model and vector database are available.

Description:

1. The user enters a natural-language question and submits it via the interface.
2. The system converts the question into an embedding vector using the embedding model.
3. The system searches the vector database for the most semantically similar chunks using the query vector.
4. The retrieved chunks are reranked so that the most relevant passages appear first.
5. If no relevant chunks are found or the similarity score is below the defined threshold, the abstention flow in F60 is triggered.
6. Otherwise, the relevant chunks are passed to F50 for answer generation.

Postconditions (Success): A ranked list of relevant chunks is returned and passed to F50.

Postconditions (Failure): No relevant chunks are found. The abstention flow in F60 is triggered.

Analysis of Function F50: Answer Generation with Citations (RM2, RS2):

Use Case: Generate a grounded answer with citations based on the retrieved chunks.

Actors: System

Trigger: Relevant document chunks have been retrieved and ranked by F40. Preconditions:

Relevant chunks with sufficient similarity scores are available from F40. The language model (LLM) is available and operational.

Description:

1. The system passes the user query together with the relevant chunks as context to the LLM.

2. The LLM generates an answer strictly based on the provided context, without adding unsupported information.
3. The system extracts citation information (source document name and page number) from the chunks used.
4. The answer is displayed together with the corresponding citations in a clear and readable format.
5. If the retrieved chunks do not contain sufficient evidence to generate a grounded answer, the abstention flow in F60 is triggered instead.

Postconditions (Success): A grounded answer with source document name and page number is displayed to the user.

Postconditions (Failure): Insufficient evidence is detected. The abstention flow in F60 is triggered.

Analysis of Function F60: Abstention and Error Handling (RM3, RM4):

Use Case: Inform the user when no sufficient evidence is found or when a system error occurs.

Actors: System, User

Trigger: Either no relevant chunks were found in F40, evidence was insufficient in F50, or a system error occurred during any processing step. Preconditions:

The retrieval step in F40 returned no relevant results, or the confidence score is below the defined threshold, or a system error has occurred.

Description:

1. If no relevant chunks were found or the similarity score is below the threshold, the system does not forward the query to the LLM.

2. Instead, the system directly displays a clear abstention message to the user (e.g. "No sufficient evidence was found in the uploaded documents").
3. If a system error occurs during any step (e.g. PDF parsing, embedding generation), the system catches the exception and displays an understandable error message.
4. In both cases, the system remains fully operational and does not crash.

Postconditions (Success – Abstention): A clear abstention message is displayed. No unsupported answer is generated.

Postconditions (Success – Error): A clear error message is displayed. The system remains stable and operational.

Analysis of Function F70: Evaluation Pipeline and Metrics Analysis (RS2)

Use Case: Evaluation of the RAG system

Requirement: The system shall evaluate the performance of the retrieval-augmented generation pipeline using predefined datasets and evaluation metrics.

Objective: Measure the quality, reliability, and effectiveness of the system and identify potential improvements.

Preconditions:

- Indexed documents are available
- Evaluation dataset containing questions and reference answers is available

Postconditions (Success):

- Evaluation metrics are calculated
- Results are stored and displayed in the evaluation dashboard

Postconditions (Failure):

- Evaluation process is aborted
- Error message is logged and displayed

Actors:

- Administrator
- System

Trigger: Administrator starts an evaluation run.

Description:

1. Load evaluation dataset
2. Execute retrieval for each evaluation query
3. Generate answers using the RAG pipeline
4. Compare generated answers with reference answers
5. Calculate evaluation metrics:
 - Retrieval Precision
 - Retrieval Recall
 - Answer Relevancy
 - Faithfulness
 - Citation Correctness
 - Abstention Accuracy
6. Store evaluation results
7. Display results in the evaluation dashboard

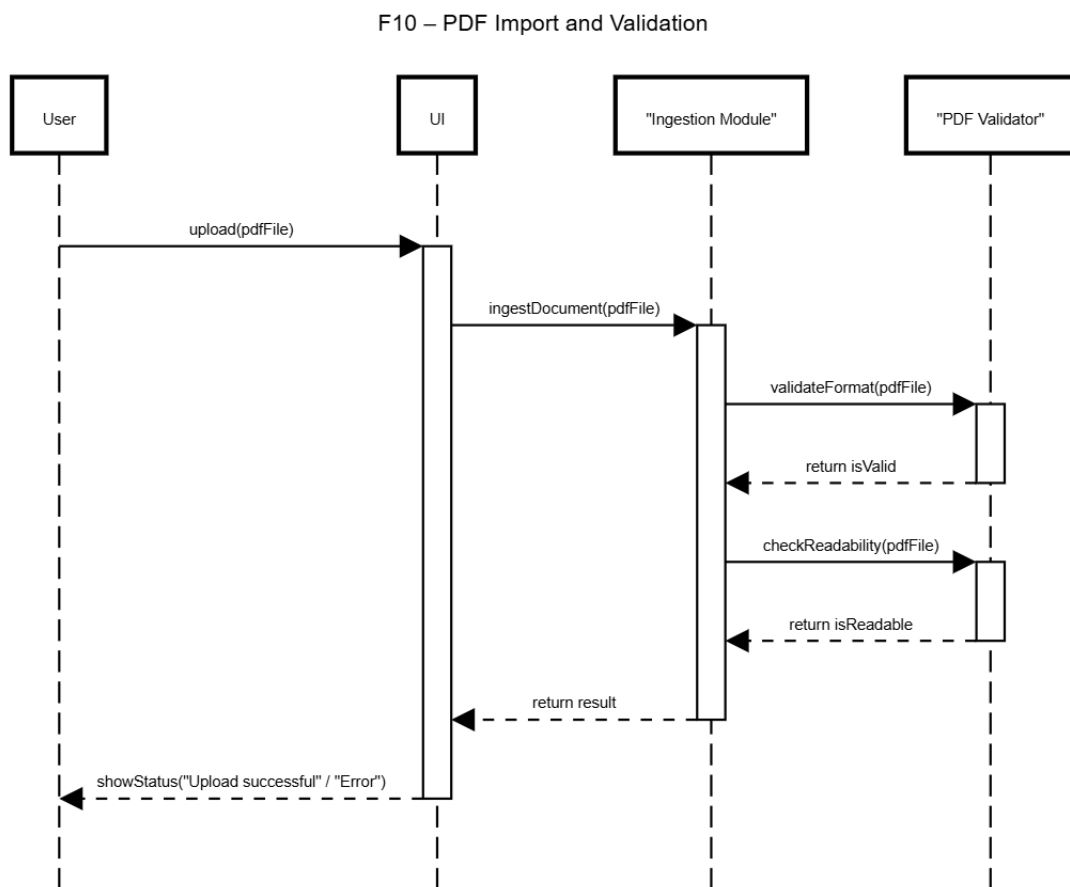


Figure 3.1: Sequenzdiagramm F10

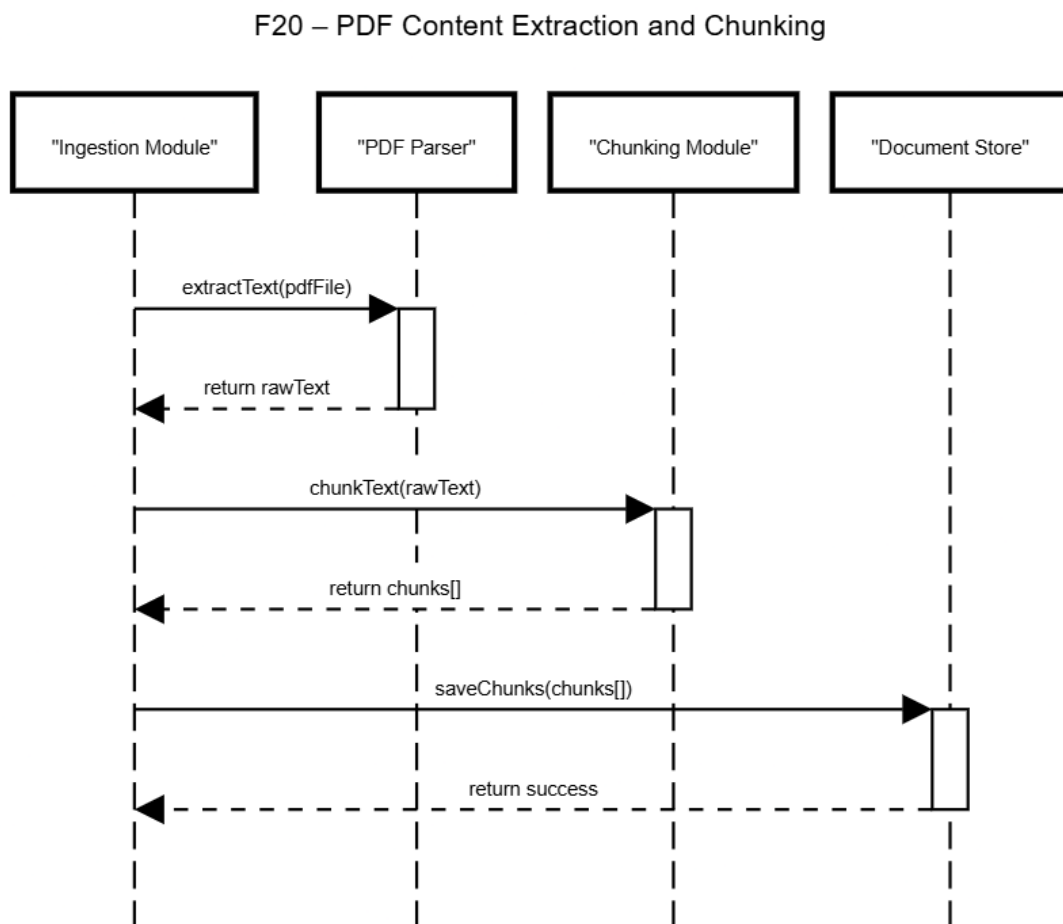


Figure 3.2: Sequenzdiagramm F20

F30 – Embedding Generation and Indexing

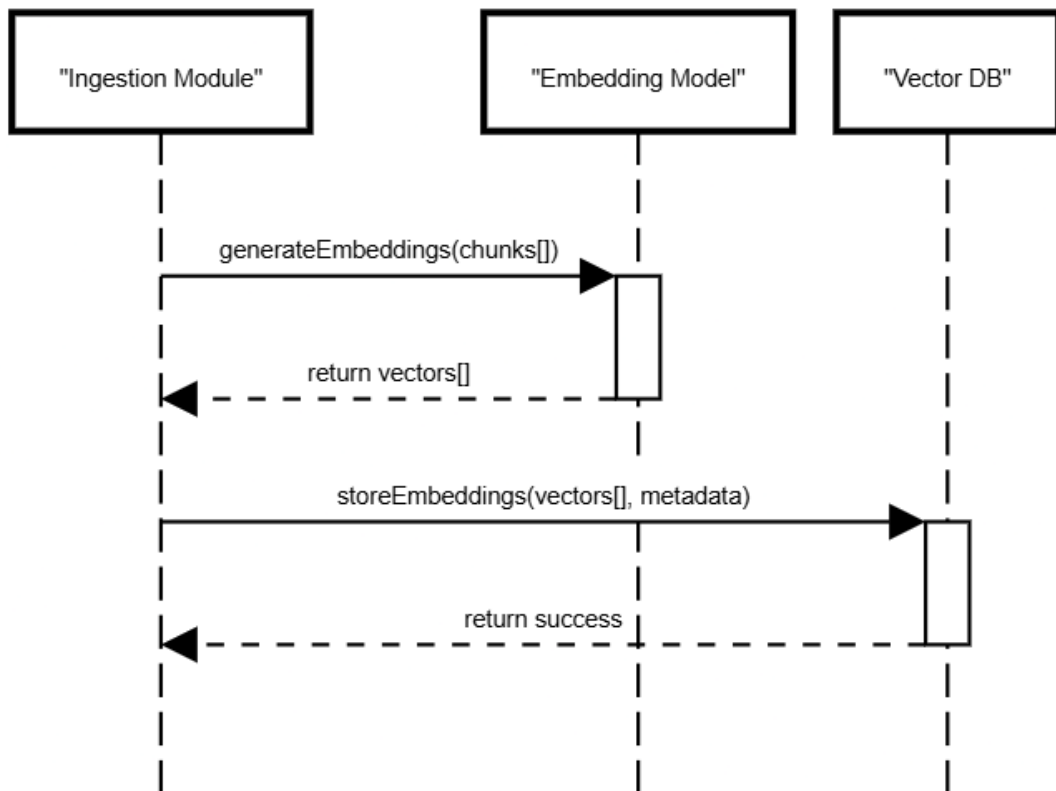


Figure 3.3: Sequenzdiagramm F30

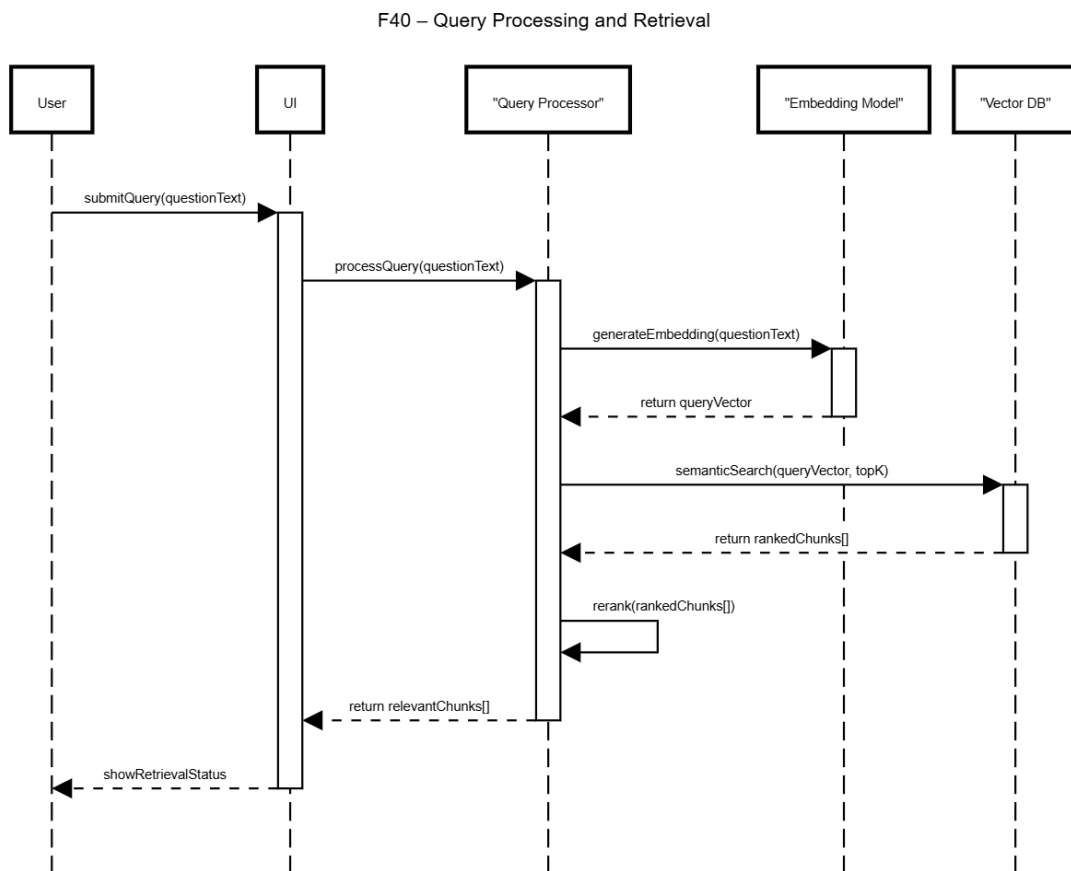


Figure 3.4: Sequenzdiagramm F40

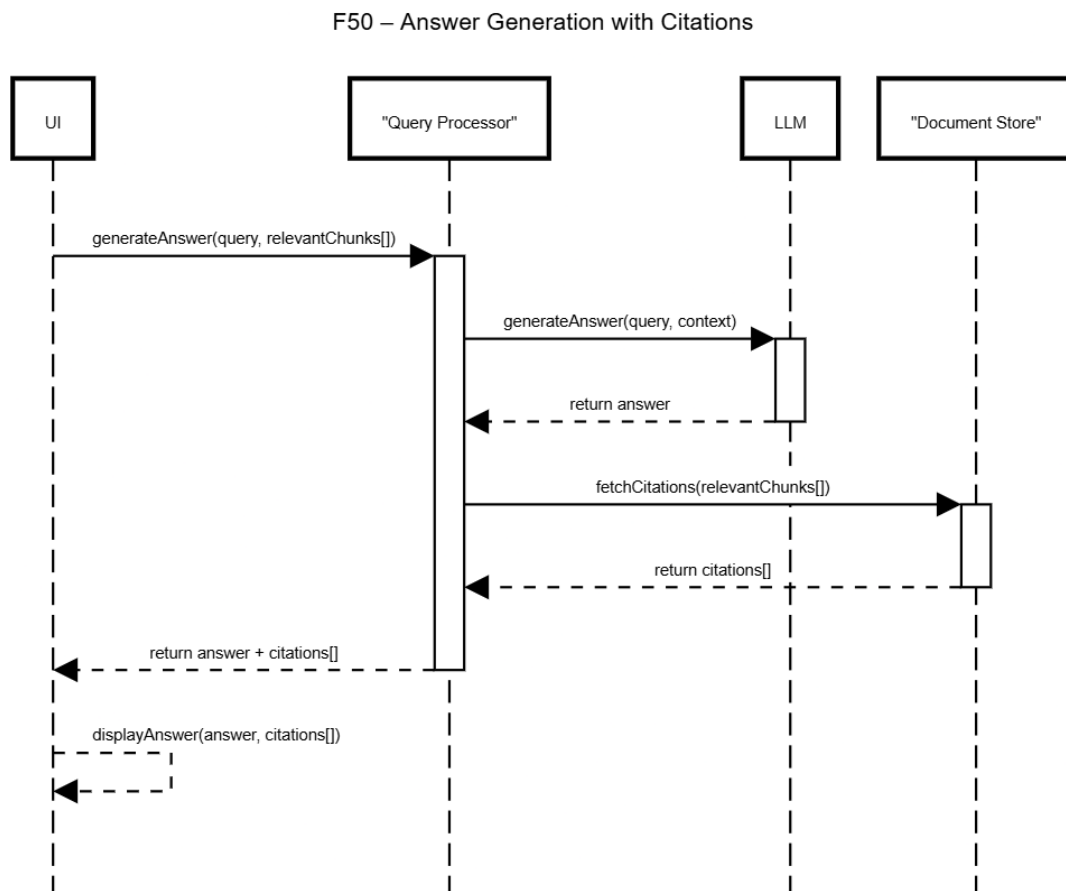


Figure 3.5: Sequenzdiagramm F50

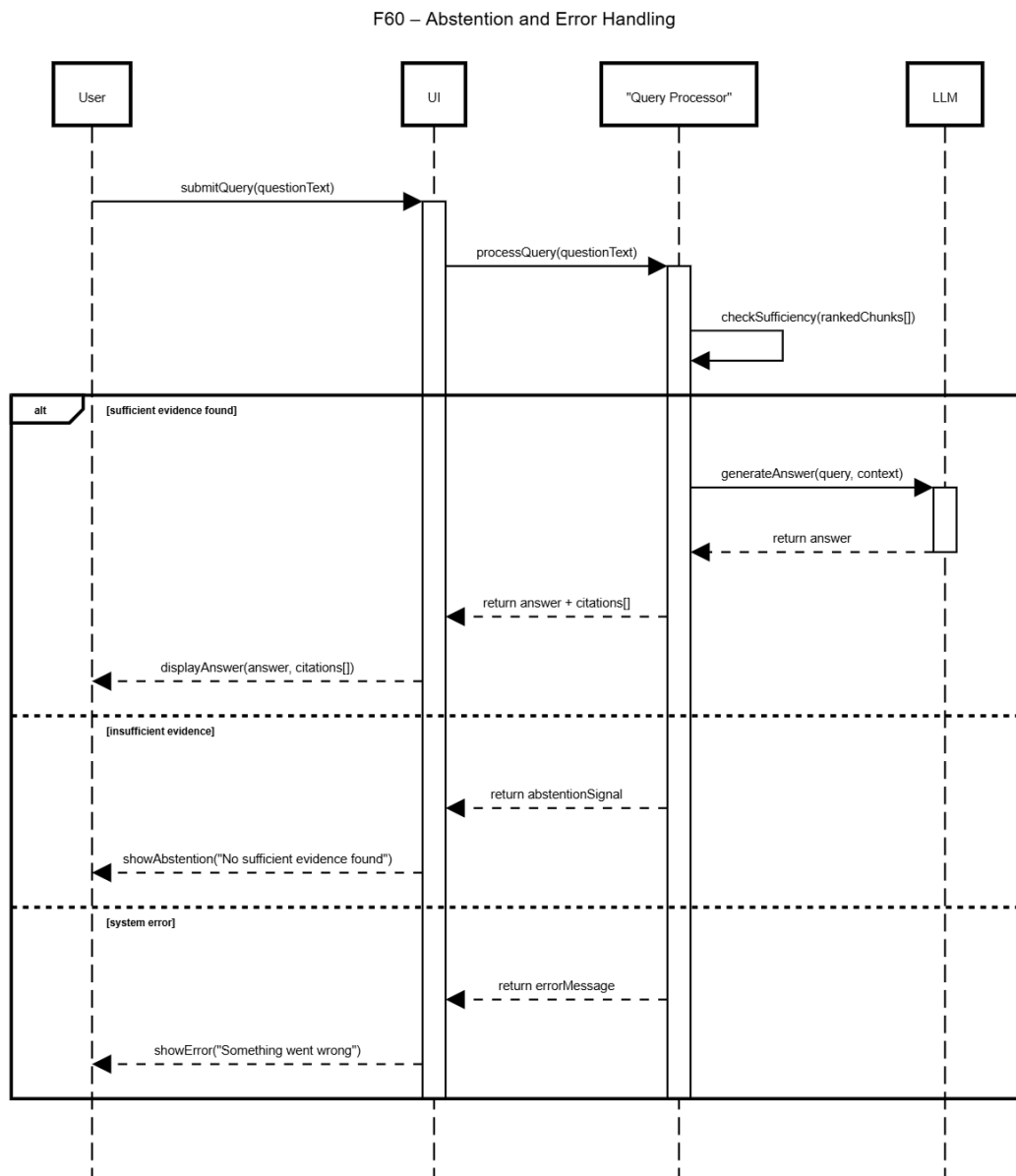


Figure 3.6: Sequenzdiagramm F60

4 Resulting Software Architecture

The architecture is derived from the sequence diagrams of the functional analysis (Chapter 2). The system is divided into six components: one user-facing component, three components responsible for the core processing logic, and two shared infrastructure components used by multiple other components. In addition, the system relies on one external dependency: a local language model Llama 3.2 running via a local Ollama instance on `localhost:11434`, which is not implemented by the project itself but is accessed through a defined interface.

4.1 Component Specification

Komponente $\langle C10 \rangle$: User Interface

Provides the main interaction point for both regular users and administrators. Displays the document upload area, the chat-based question-answering view, and the administration and evaluation dashboard. Forwards user actions such as document upload, question submission, and evaluation start to the responsible backend components and displays their results, including grounded answers with citations, abstention messages, and error messages. Implemented as a Streamlit web application in `streamlit_app.py`.

Communication partners: Document Ingestion, Retrieval Engine, Evaluation Module

Komponente $\langle C20 \rangle$: Document Ingestion

Validates uploaded PDF files, extracts their textual content page by page, and splits the extracted text into overlapping chunks. Forwards the chunks to the Embedding Service for embedding generation and stores the resulting chunks together with their metadata in the Data Storage component. Implemented across `parser.py`, `chunker.py`, and `indexer.py`.

Communication partners: User Interface, Embedding Service, Data Storage

Komponente $\langle C30 \rangle$: Retrieval Engine

Processes natural-language questions submitted by the user. Converts the question into an embedding via the Embedding Service, retrieves the most relevant chunks from Data Storage using cosine similarity, and checks whether sufficient evidence is available by comparing the similarity score against a configurable abstention threshold. If sufficient evidence exists, it forwards the question and the relevant chunks to the local LLM to generate a grounded answer with citations. If not, it triggers the abstention flow instead of contacting the LLM. Implemented in `retriever.py`, `generator.py`, and `pipeline.py`.

Communication partners: User Interface, Embedding Service, Data Storage, Local LLM

Komponente $\langle C40 \rangle$: Evaluation Module

Executes the automated evaluation pipeline. Loads a benchmark dataset of questions and reference answers from the Open RAG Benchmark, runs each query through the Retrieval Engine, compares the generated answers with the reference answers, and computes evaluation metrics such as retrieval precision, retrieval recall, answer relevancy, faithfulness, citation correctness, and abstention accuracy. Stores and displays the results in the administration dashboard. Implemented in the `evaluation/` directory.

Communication partners: User Interface, Retrieval Engine, Data Storage

Komponente $\langle C50 \rangle$: Embedding Service

Generates numerical vector representations (embeddings) for a given text input. It is used both during document ingestion to embed text chunks and during query processing to embed the user question, so that semantic similarity can be computed between the two. Implemented using the Sentence Transformers model `all-MiniLM-L6-v2`, which produces 384-dimensional vectors.

Communication partners: Document Ingestion, Retrieval Engine

Komponente $\langle C60 \rangle$: Data Storage

Persistently stores all document-related data: document metadata, text chunks, their embeddings, and the associated vector index used for similarity search. Provides search functionality through semantic similarity search as well as direct lookup of chunks and citations by ID. Implemented using ChromaDB as the local vector database.

Communication partners: Document Ingestion, Retrieval Engine



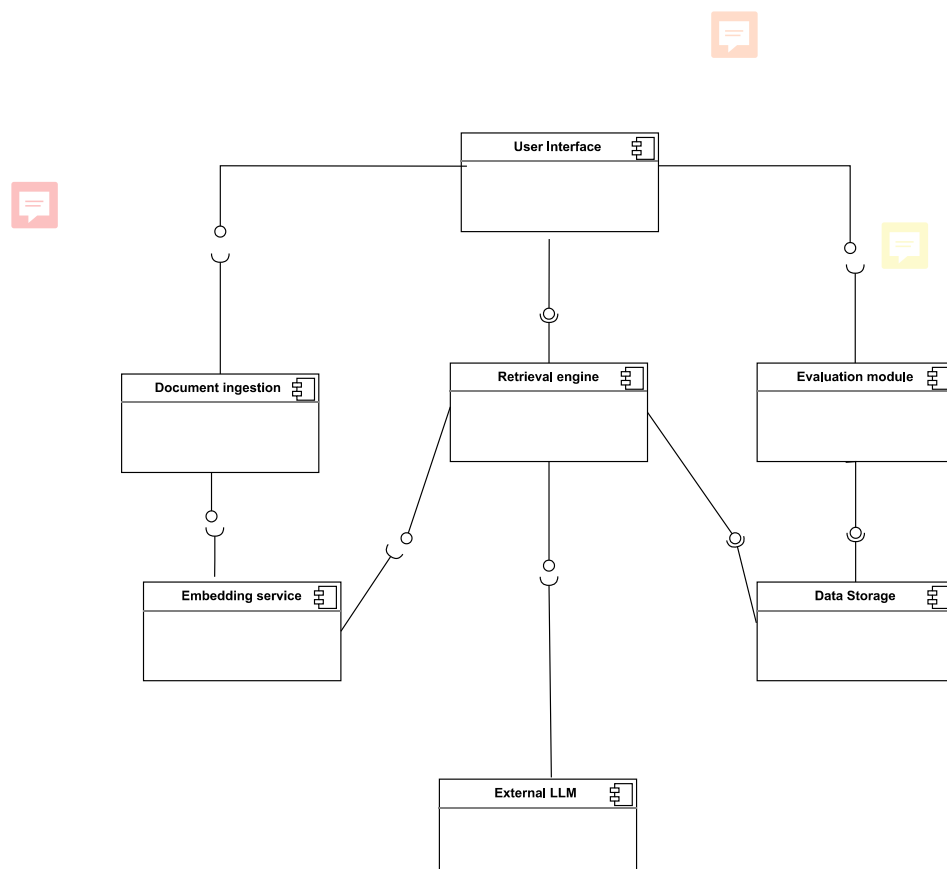


Figure 4.1: Component Diagram

4.2 Interface Specification



4.2.1 Interfaces

Interface $\langle I10 \rangle$: IngestionInterface

Offered by: Document Ingestion

Operation	Description
<code>ingestDocument(file: PDFFile) : IngestionResult</code>	Validates, extracts, and chunks the uploaded PDF file. Returns the processing status (success, validation error, or extraction error).

Interface $\langle I20 \rangle$: EmbeddingInterface

Offered by: Embedding Service

Operation	Description
<code>generateEmbeddings(chunks: Chunk[]) : Vector[]</code>	Generates one 384-dimensional embedding vector per text chunk using all-MiniLM-L6-v2. Used during document indexing.
<code>generateEmbedding(text: String) : Vector</code>	Generates a single 384-dimensional embedding vector for a user query. Used during retrieval.

Interface $\langle I30 \rangle$: StorageInterface

Offered by: Data Storage

Operation	Description
<code>saveChunks(chunks: Chunk[]) : Boolean</code>	Persists text chunks with their metadata including document name and page number. Returns True on success, False on failure.
<code>storeEmbeddings(vectors: Vector[], metadata: Metadata) : Boolean</code>	Persists embedding vectors together with the corresponding chunk metadata in ChromaDB. Returns True on success, False on failure.
<code>semanticSearch(queryVector: Vector, topK: Integer) : Chunk[]</code>	Returns the top-k most similar chunks for a given query embedding using cosine similarity search in ChromaDB.
<code>fetchCitations(chunkIds: String[]) : Citation[]</code>	Returns document name and page number for a given set of chunk IDs.

Interface **<I40>: RetrievalInterface**

Offered by: Retrieval Engine

Operation	Description
<code>processQuery(questionText: String) : AnswerResult</code>	Processes a natural-language question and returns either a grounded answer with citations or an abstention message. The AnswerResult contains the answer text, the list of citations, the abstention flag, and the response time in milliseconds.

Interface **<I50>: EvaluationInterface**

Offered by: Evaluation Module

Operation	Description
<code>runEvaluation(dataset: Dataset) : EvaluationReport</code>	Executes the evaluation pipeline on a given benchmark dataset and returns the computed metrics including faithfulness, answer relevancy, context recall, context precision, abstention rate, citation correctness, and average latency.
<code>getEvaluationResults() : EvaluationReport</code>	Returns the most recently stored evaluation results for display in the administration dashboard.

Interface `<I60>`: Local LLM Interface

Offered by: Local Ollama Instance, used by Retrieval Engine

Operation	Description
<code>generateAnswer(query: String, context: Chunk[]) : String</code>	Generates an answer strictly based on the provided context chunks without adding unsupported information. Called via the <code>ollama</code> Python library on <code>localhost:11434</code> using the model <code>llama3.2</code> .

4.3 Protocols for the Use of the Components

Of the six components, three follow interaction patterns that are worth documenting with a protocol statechart: the Embedding Service, the Data Storage component, and the **Retrieval Engine**.

The Embedding Service and Data Storage are generic enough to be reusable independently of this specific project. Both expose a stable, narrow interface that could be reused in a different RAG-based application with minimal changes.

The Retrieval Engine follows an interesting state protocol because it involves an internal decision that determines whether the LLM is called or not, which directly affects system reliability and abstention behavior.

The remaining components, namely User Interface, Document Ingestion, and Evaluation Module, are tightly coupled to the specific business logic of this project and are therefore not considered for reuse. Their usage follows a simple request-response pattern without meaningful internal states, so a protocol statechart would not provide additional value.

Protocol for Component <C50>: Embedding Service The component starts in state *Idle*. Upon receiving a generation request (`generate(text)`), it transitions to *Generating*. On success, it moves to *Ready* and returns the embedding vectors. On failure, for example if the Sentence Transformers model is unavailable, it transitions to *Error*. Both *Ready* and *Error* return to *Idle* once the result has been delivered to the caller. The component must not accept a new request while it is in state *Generating*.

Protocol for Component <C60>: Data Storage The component is in state *Idle* when no operation is active. A save or store call transitions it to *Writing*. A search or fetch call transitions it to *Reading*. Both states return to *Idle* upon successful completion. If the underlying ChromaDB storage is unavailable, the component transitions from either state to *Error* and returns to *Idle* once the error has been reported to the caller.

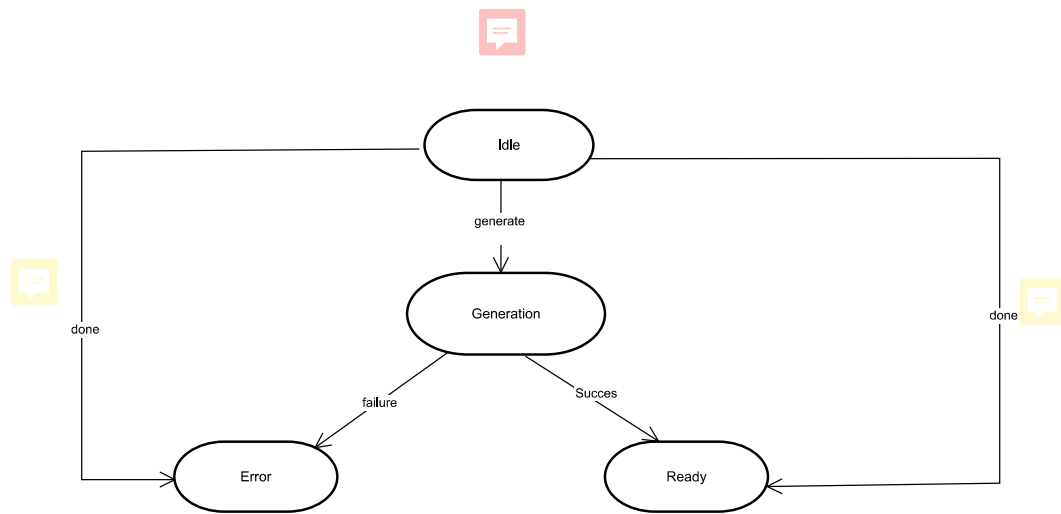


Figure 4.2: Protocol Statechart: Embedding Service (C50)

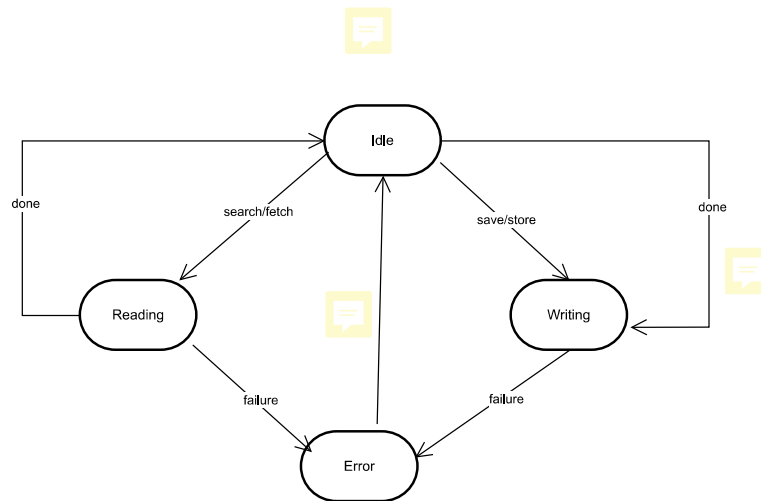


Figure 4.3: Protocol Statechart: Data Storage ⟨C60⟩

5 Verteilungsentwurf

The PDF Search Agent is implemented as a distributed system. The client application runs on the user's local workstation, while the AI services are executed on a GPU compute node of the IAI cluster. Communication between the local workstation and the cluster is established via an SSH tunnel through the login node. The Ollama server, language model, and embedding model run inside an Apptainer container on the GPU compute node. Figure 5.1 illustrates the deployment architecture of the system.

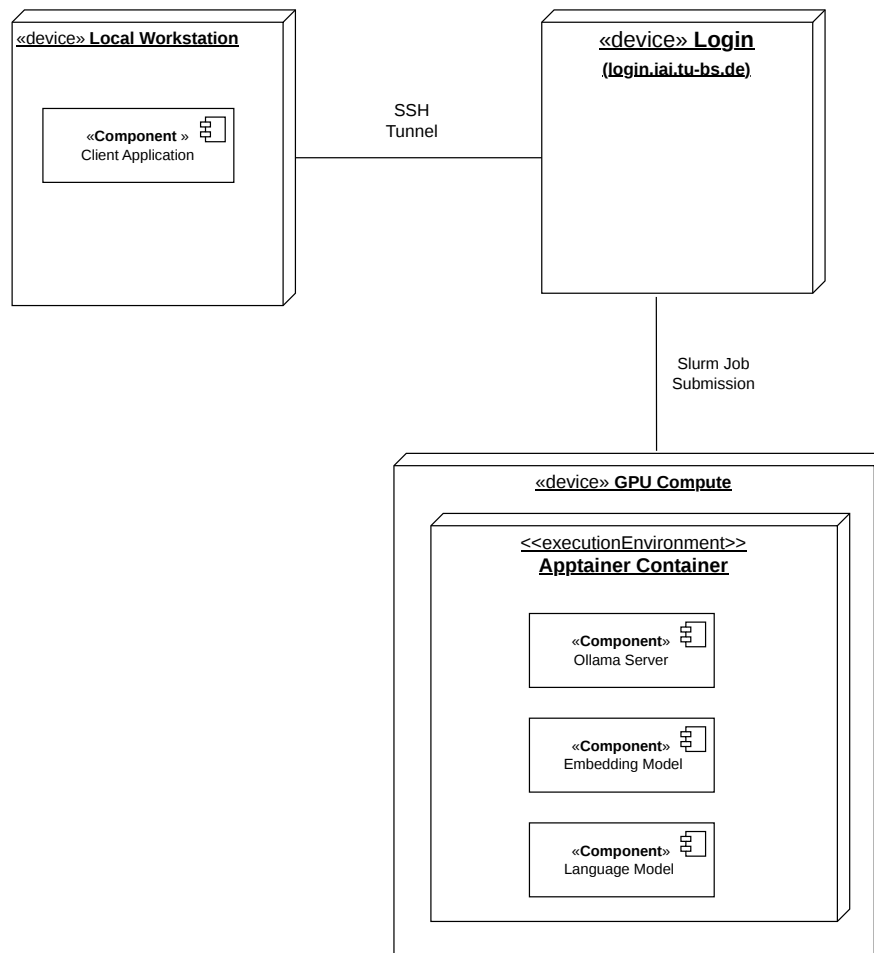


Figure 5.1: Deployment Diagram of the PDF Search Agent

6 Implementation Design

This chapter documents the implementation of each system component defined in Chapter 3. For each component, the classes, libraries, design decisions, and key operations are described. Class diagrams illustrate the structure of each component where applicable.

Note on component mapping: The architecture defined in Chapter 3 describes six logical components. In the actual implementation, the Embedding Service (C50) is not a standalone module but is integrated into the Indexer class inside `app/indexer.py`, which also represents the core of Data Storage (C60). Document Ingestion (C20) therefore spans three files: `parser.py`, `chunker.py`, and `indexer.py`. This is a deliberate design decision to keep the implementation simple and avoid unnecessary abstraction layers.

6.1 Implementation of Component (C10): User Interface

The User Interface component is implemented as a Streamlit web application in `ui/streamlit_app.py`. Streamlit was chosen because it allows building interactive web applications entirely in Python without requiring a separate frontend framework. The interface uses Streamlit session state to manage uploaded documents, chat history, and evaluation logs across user interactions.

The component is structured around two main tabs: the Search tab, which provides the document upload area and the chat-based question-answering interface, and the Administration and Evaluation tab, which displays system statistics and query logs.

The current implementation is a working frontend prototype. PDF validation is performed directly in the `process_pdf` function by checking the file extension, file size, and PDF header bytes. Answer generation is currently handled by a keyword-based demo function that returns predefined responses. In the target implementation, `process_pdf` will call `pipeline.processPdf()`

and `generate_answer` will call `pipeline.answerQuestion()` to connect the interface to the real backend.

The design follows a simple event-driven pattern: user actions such as file upload, question submission, and session reset trigger state updates and rerenders via `st.rerun()`.

This component does not follow an object-oriented design and consists entirely of standalone functions. The diagram in Figure 6.1 gives a structured overview of all functions and their relationships within the component.

6.1.1 Class Diagram

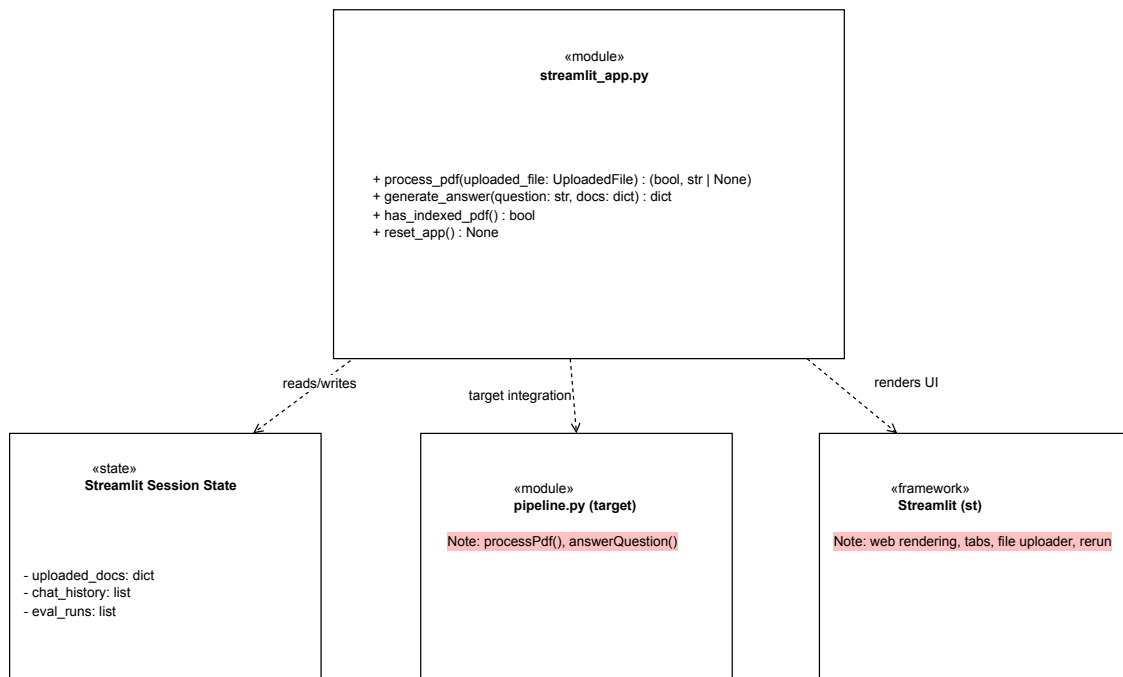


Figure 6.1: Class Diagram: User Interface (C10)

Libraries used:

- **streamlit**: web application framework
- **datetime**: timestamp generation for evaluation logs
- **time**: response time measurement

6.1.2 Description



Function	<code>process_pdf(uploaded_file) : (bool, str None)</code>
Task	Validates an uploaded file and registers it in the Streamlit session state. Checks that the file ends with <code>.pdf</code> , is not empty, and starts with the <code>%PDF</code> header. In the target implementation, will call <code>pipeline.processPdf()</code> for real parsing and indexing.
Returns	True on success, False with error message on failure
Communication	Streamlit session state, Pipeline (target)

Function	<code>generate_answer(question: str, docs: dict) : dict</code>
Task	Currently returns keyword-based demo responses. In the target implementation, will call <code>pipeline.answerQuestion()</code> and return the real backend result.
Returns	Dict with <code>text</code> , <code>citations</code> , <code>abstain</code> , <code>error</code>
Communication	Streamlit session state, Pipeline (target)

Function	<code>has_indexed_pdf() : bool</code>
Task	Checks whether at least one document with status <i>completed</i> exists in the session state. Used as a guard before allowing question submission.
Returns	True if at least one indexed document is available
Communication	Streamlit session state

Function	<code>reset_app() : None</code>
Task	Resets the complete session state by removing all uploaded documents and clearing the chat history and evaluation log.
Returns	None
Communication	Streamlit session state

6.2 Implementation of Component **⟨C20⟩**: Document Ingestion

The Document Ingestion component is implemented across three modules: `app/parser.py`, `app/chunker.py`, and `app/indexer.py`. Together they form the complete document processing pipeline from PDF upload to vector storage. Note that `indexer.py` also represents the implementation of the Embedding Service **⟨C50⟩** and Data Storage **⟨C60⟩**, as described at the beginning of this chapter.

Libraries used:

- `fitz` (PyMuPDF): PDF text extraction in `parser.py`
- `chromadb`: vector storage and collection management in `indexer.py`
- `sentence_transformers`: embedding generation via `SentenceTransformerEmbeddingFunction` in `indexer.py`
- `uuid`: unique ID generation for documents and chunks
- `dataclasses`: structured data classes for `DocumentData` and `TextChunk`
- `logging`: pipeline logging across all three modules

6.2.1 Class Diagram

6.2.2 Description

Class	<code>DocumentData : parser.py</code>
Task	Pure data container storing metadata about an uploaded PDF. Implemented as a Python dataclass with no methods.
Attributes	<code>document_id: str</code> : unique UUID generated on upload <code>file_name: str</code> : original file name <code>upload_date: str</code> : ISO-8601 timestamp <code>file_size: int</code> : size in bytes <code>file_type: str</code> : always <code>application/pdf</code> <code>storage_path: str</code> : absolute path on disk
Operations	none (dataclass)
Communication	Created by <code>PDFParser</code> , received by <code>parseAndChunk</code>

Class	<code>PDFParser : parser.py</code>
Task	Validates uploaded PDF files and extracts text content page by page using <code>PyMuPDF</code> . Raises <code>FileNotFoundError</code> if the file does not exist, and <code>ValueError</code> if the file is invalid, empty, or unreadable. Empty pages are skipped and text is cleaned by removing extra whitespace.
Attributes	none
Operations	<code>parse(file_path: str) : tuple[DocumentData, dict[int, str]]</code> Creates <code>DocumentData</code> , opens PDF with <code>fitz.open()</code> , extracts text per page using <code>page.get_text(ttext="")</code> , returns metadata and page texts. <code>parse_multiple(file_paths: list[str]) : list[tuple]</code> Calls <code>parse()</code> on each file, skips invalid ones with a warning.
Communication	Called by <code>parseAndChunk</code> , uses <code>fitz</code> library

Class	<code>TextChunk : chunker.py</code>
Task	Represents a single text segment produced during chunking. Stores all metadata required for retrieval and citation generation. Implemented as a Python dataclass.
Attributes	<code>chunk_id: str</code> : unique UUID <code>document_id: str</code> : reference to parent <code>DocumentData</code> <code>text_content: str</code> : the chunk text <code>chunk_index: int</code> : position within the document <code>token_count: int</code> : approximate word count <code>page_number: int</code> : source page number for citations <code>embedding_vector: list</code> : empty at creation, filled by <code>Indexer</code>
Operations	none (dataclass)
Communication	Created by chunkers, consumed by <code>Indexer</code> , references <code>DocumentData</code> via <code>document_id</code>

Class	<code>FixedSizeChunker : chunker.py</code>
Task	Splits text into fixed-size word windows. Chunks with fewer than 20 words are skipped.
Attributes	<code>chunk_size: int = 400</code> : maximum words per chunk <code>overlap: int = 50</code> : shared words between neighbors <code>MIN_WORDS: int = 20</code> : minimum chunk size
Operations	<code>chunk(text: str, page_number: int, document_id: str, start_index: int) : list[TextChunk]</code> Splits text into windows of <code>chunk_size</code> words with <code>step = chunk_size - overlap</code> .
Communication	Called by <code>parseAndChunk</code> , creates <code>TextChunk</code> objects

Class	<code>RecursiveChunker : chunker.py</code>
Task	Splits text recursively using a separator hierarchy: paragraph breaks, line breaks, sentence endings, and word spaces. More retrieval-friendly than fixed-size chunking. Used as the default strategy.
Attributes	<code>chunk_size: int = 400</code> <code>overlap: int = 50</code> <code>SEPARATORS: list[str] : ["\n\n", "\n", ". ",]</code> <code>MIN_WORDS: int = 20</code>
Operations	<code>chunk(...) : list[TextChunk] : main entry point</code> <code>_split_text(text, separators) : list[str] : recursive split</code> <code>_merge_with_overlap(segments) : list[str] : merge with overlap</code> <code>_hard_split(text) : list[str] : word-count fallback</code>
Communication	Called by <code>parseAndChunk</code> as default strategy, creates <code>TextChunk</code> objects

Function	<code>parseAndChunk(file_path: str, chunker: FixedSizeChunker RecursiveChunker None) :</code> <code>list[TextChunk] : chunker.py</code>
Task	Main pipeline entry point for document ingestion. Calls <code>PDFParser.parse()</code> to extract text, then applies the selected chunking strategy page by page. Maintains a global chunk index across all pages. Uses <code>RecursiveChunker</code> by default.
Communication	Calls <code>PDFParser</code> and the selected chunker, creates <code>TextChunk</code> objects, called by <code>Indexer</code> and <code>Pipeline</code>

Class	<code>Indexer : indexer.py</code>
Task	Converts text chunks into 384-dimensional embedding vectors and stores them in ChromaDB. Also provides semantic search for the Retriever. Implements both $\langle C50 \rangle$ Embedding Service and $\langle C60 \rangle$ Data Storage.
Attributes	<code>embedding_fn: SentenceTransformerEmbeddingFunction using all-MiniLM-L6-v2</code> <code>client: chromadb.PersistentClient at ./data/chroma_db</code> <code>collection: ChromaDB collection named pdf_chunks</code> <code>collection_name: str = "pdf_chunks"</code> <code>persist_directory: str = "./data/chroma_db"</code>
Operations	<code>__init__(collection_name, persist_directory) : None</code> : sets up client and collection <code>index_chunks(chunks: list[TextChunk]) : None</code> : stores chunks in ChromaDB with auto-generated embeddings <code>search(query: str, top_k: int) : list[dict]</code> : embeds query and returns top-k similar chunks <code>clear() : None</code> — removes all chunks from the collection
Communication	Receives <code>TextChunk</code> objects from <code>parseAndChunk</code> , provides <code>search()</code> to Retriever

6.3 Implementation of Component $\langle C30 \rangle$: Retrieval Engine

The Retrieval Engine component is implemented in `app/retriever.py`, `app/generator.py`, and `app/pipeline.py`. The Retriever handles semantic search by delegating to the Indexer. The Generator handles prompt construction, LLM communication, abstention detection, and citation extraction. The Pipeline orchestrates the complete question-answering workflow and serves as the single entry point called by the User Interface.

Libraries used:

- `ollama`: Python library for communicating with the local Llama 3.2 model via `localhost:11434`



- logging: pipeline logging across all three modules

6.3.1 Description

Class	<code>Retriever : retriever.py</code>
Task	Performs semantic similarity search over indexed document chunks by delegating to <code>Indexer.search()</code> . Returns an empty list immediately for empty queries.
Attributes	<code>indexer: Indexer</code> : injected dependency from <code><C20></code>
Operations	<code>__init__(indexer: Indexer) : None</code> : stores the Indexer reference <code>retrieve(query: str, top_k: int = 5) : list[dict]</code> : calls <code>indexer.search()</code> , returns top-k chunks with text, document ID, page number, and similarity score
Communication	Called by <code>answerQuestion</code> , delegates to <code>Indexer.search()</code>

Function	<code>generateAnswer(question: str, chunks: list) : dict</code> — <code>generator.py</code>
Task	Main answer generation function. Returns abstention immediately if no chunks are provided. Otherwise calls <code>buildContext</code> , builds a structured prompt, sends it to Llama 3.2 via <code>ollama.chat()</code> , checks for abstention phrase <i>could not find</i> , and calls <code>extractCitations</code> if the answer is valid.
Returns	Dict with <code>answer</code> , <code>citations</code> , <code>abstained</code>
Communication	Calls <code>buildContext</code> and <code>extractCitations</code> , calls <code>ollama.chat()</code> on <code>localhost:11434</code> , called by <code>answerQuestion</code>

Function	<code>buildContext(chunks: list) : str : generator.py</code>
Task	Formats retrieved chunks into a structured context string for the LLM prompt. Labels each chunk with source and page using the format <code>[Source: name, Page: N]</code> .
Returns	Formatted context string
Communication	Called by <code>generateAnswer</code>

Function	<code>extractCitations(chunks: list) : list : generator.py</code>
Task	Extracts unique citations from retrieved chunks. Deduplicates by combining source name and page number into a key and using a seen-set.
Returns	List of unique dicts with <code>source</code> and <code>page</code>
Communication	Called by <code>generateAnswer</code>

Function	<code>processPdf(filePath: str) : dict : pipeline.py</code>
Task	Orchestrates the full document ingestion workflow by calling <code>parseAndChunk</code> and then <code>index_chunks</code> . Falls back gracefully with a logged warning if either module is not available.
Returns	Dict with <code>success</code> flag and <code>message</code> string
Communication	Calls <code>parseAndChunk</code> from <code>chunker.py</code> , calls <code>index_chunks</code> from <code>indexer.py</code>

Function	<code>answerQuestion(question: str, topK: int = 5) : dict — pipeline.py</code>
Task	Orchestrates the full question-answering workflow. Calls <code>Retriever.retrieve()</code> , normalizes chunks via <code>convertChunks</code> , and calls <code>generateAnswer</code> . Catches all exceptions and returns an error abstention result if something goes wrong.
Returns	Dict with <code>answer</code> , <code>citations</code> , <code>abstained</code>
Communication	Calls <code>Retriever.retrieve()</code> , <code>convertChunks</code> , and <code>generateAnswer</code>

Function	<code>convertChunks(rawChunks: list) : list : pipeline.py</code>
Task	Normalizes raw chunks from the Retriever into the unified format expected by the Generator. Handles both <code>TextChunk</code> dataclass objects and plain dictionaries transparently.
Returns	Normalized list of dicts with <code>text</code> , <code>source</code> , <code>page</code>
Communication	Receives chunks from <code>Retriever</code> , passes normalized chunks to <code>generateAnswer</code>

6.4 Implementation of Component **⟨C40⟩**: Evaluation Module

The Evaluation Module is not yet fully implemented. The module structure and the integration points with the Retrieval Engine are defined, and the implementation will be completed in the next development phase. The functions described below represent the intended design based on the current project plan. No class diagram is provided for this component since no classes have been implemented yet.

Libraries intended for use:

- `ragas`: evaluation framework for RAG systems
- `datasets`: HuggingFace datasets library for loading the Open RAG Benchmark
- `json`: storing evaluation results
- `csv`: storing evaluation results in tabular format

6.4.1 Description

Function	<code>load_benchmark(subset_size: int) : list[dict] : benchmark_loader.py</code>
Task	Is intended to load a defined subset of QA pairs from the Open RAG Benchmark using the HuggingFace <code>datasets</code> library. Returns dicts with <code>question</code> , <code>ground_truth</code> , <code>source_pdf</code> , and <code>source_page</code> . A local fallback allows offline use on the IAI cluster.
Communication	Called by <code>run_evaluation</code>

Function	<code>compute_ragas_scores(evaluation_results: list[dict]) : dict : eval_ragas.py</code>
Task	Is designed to pass stored prediction results to the RAGAS framework and compute faithfulness, answer relevancy, context recall, and context precision automatically.
Communication	Called by <code>run_evaluation</code> , uses <code>ragas</code> library

Function	<code>compute_abstention_rate(results: list[dict]) : float : metrics.py</code>
Task	Is designed to compute the proportion of questions for which the system correctly abstained. Returns a value between 0 and 1.
Communication	Called by <code>run_evaluation</code>

Function	<code>compute_citation_correctness(results: list[dict]) : float : metrics.py</code>
Task	Is designed to compute the proportion of answers where the cited page number matches the expected source page from the benchmark. Returns a value between 0 and 1.
Communication	Called by <code>run_evaluation</code>

Function	<code>compute_average_latency(results: list[dict]) : float : metrics.py</code>
Task	Is designed to compute the average response time across all evaluated queries. Returns the average latency in seconds.
Communication	Called by <code>run_evaluation</code>

Function	<code>run_evaluation(benchmark_path: str, subset_size: int, output_path: str) : dict</code> — <code>run_evaluation.py</code>
Task	Is intended to be the main orchestration function. Loads the benchmark, runs the full RAG pipeline on each question via <code>pipeline.answerQuestion()</code> , passes all results to RAGAS and the independent metrics functions, and saves the final scores to <code>evaluation/results/scores.json</code> and <code>scores.csv</code> .
Communication	Calls Benchmark Loader, Retrieval Engine, RAGAS, and all metrics functions

6.5 Implementation of Component **⟨C50⟩**: Embedding Service

As noted at the beginning of this chapter, the Embedding Service is not implemented as a standalone module. It is integrated directly into the `Indexer` class via the `ChromaDB SentenceTransformerEmbedding`. This function automatically handles embedding generation during both indexing and search, so no separate embedding call is needed anywhere in the pipeline.

The model `all-MiniLM-L6-v2` produces 384-dimensional vectors. The same model is used for both chunk indexing and query embedding, which ensures that both representations are in the same vector space and cosine similarity between them is meaningful.

No class diagram is provided for this component since it has no standalone implementation.

Design decision: Integrating the Embedding Service into the `Indexer` reduces the number of abstraction layers and avoids redundant embedding calls. The interface defined in Chapter 3 **⟨I20⟩** is fulfilled internally by `ChromaDB` without requiring a separate class or module.

6.6 Implementation of Component **⟨C60⟩**: Data Storage

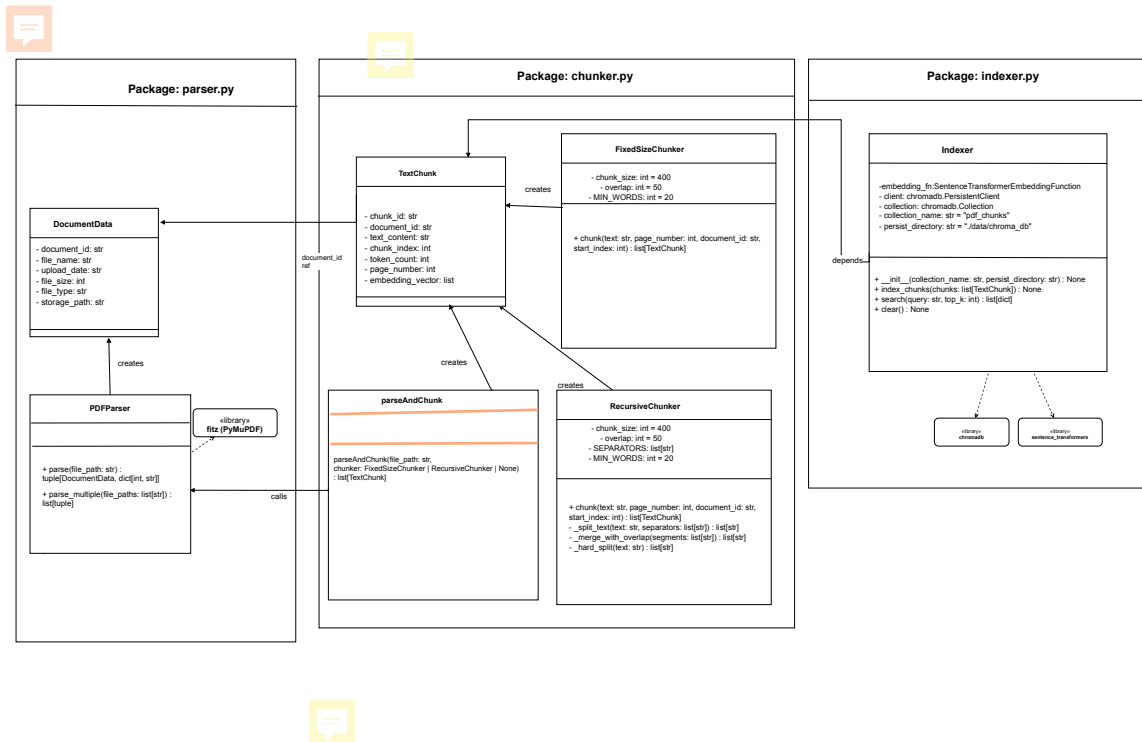
As noted at the beginning of this chapter, the Data Storage component is implemented via the `ChromaDB PersistentClient` inside `app/indexer.py`. `ChromaDB` stores all document chunks, their embedding vectors, and associated metadata persistently in a local directory at `./data/chroma_db`. No external database server or API key is required.

Data is organized in a single ChromaDB collection named `pdf_chunks`. Each entry stores the chunk text, chunk ID, document ID, page number, chunk index, and token count as metadata. Similarity search is performed directly by ChromaDB using the stored embedding vectors and the cosine distance metric.

No class diagram is provided for this component since it has no standalone implementation beyond what is already shown in the `Indexer` class in Figure 6.2.

Design decision: ChromaDB was chosen because it runs entirely locally without requiring a dedicated server, supports persistent storage across sessions, and integrates natively with Sentence Transformers embedding functions. This keeps the system fully self-contained and easy to set up on a standard development machine.

DESIGN A RELIABLE PDF SEARCH AGENT



7 Data Model

This chapter describes the persistent data used by the PDF Search Agent. The data model is based on the entities defined in the requirements specification and represents the core components of the Retrieval-Augmented Generation (RAG) pipeline.

7.1 Diagramm

Figure 7.1 shows the UML class diagram of the data model.

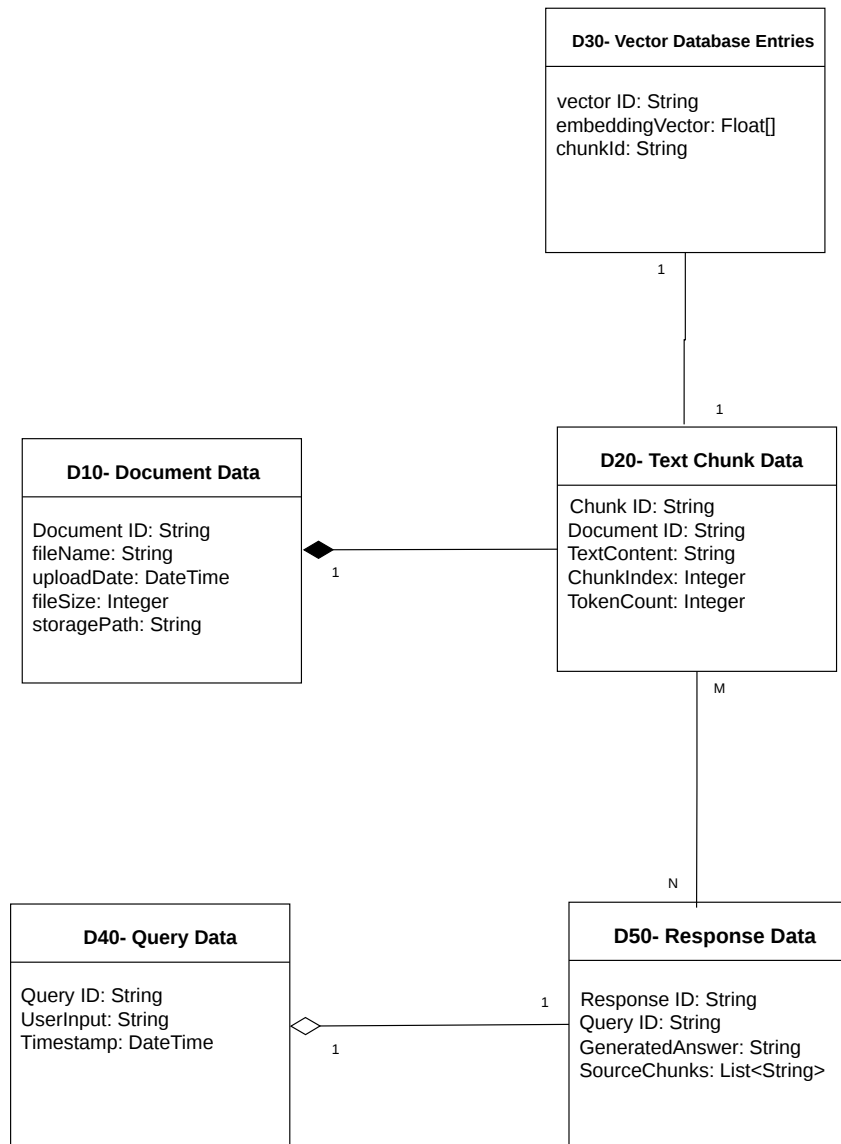


Figure 7.1: Data model of the PDF Search Agent

7.2 Explanation

The data model consists of five entities:

- Document Data (D10)
- Text Chunk Data (D20)
- Vector Database Entries (D30)
- Query Data (D40)
- Response Data (D50)

Together, these entities represent the complete flow from document upload to answer generation.

Document Data $\langle D10 \rangle$

Relationship	Cardinality	Expected Data Volume	Description
contains Text Chunks	1:n	Min: 1, Max: many	A document is split into multiple text chunks during preprocessing.

The Document entity stores metadata about uploaded PDF files.

- documentId: String
- fileName: String
- uploadDate: DateTime
- fileSize: Integer
- storagePath: String

Text Chunk Data $\langle D20 \rangle$

Relationship	Cardinality	Expected Data Volume	Description
belongs to Document	n:1	Min: 1, Max: 1	Each chunk belongs to exactly one document.
has Vector Entry	1:1	Min: 1, Max: 1	Each chunk is represented by exactly one embedding vector.
used by Response	n:m	Min: 0, Max: many	Chunks may be used as evidence in generated responses.

The Text Chunk entity represents segments extracted from PDF documents.

- chunkId: String
- documentId: String
- textContent: String
- chunkIndex: Integer
- tokenCount: Integer

The chunk index indicates the position of the chunk within the original document.

Vector Database Entries $\langle D30 \rangle$

Relationship	Cardinality	Expected Data Volume	Description
belongs to Text Chunk	1:1	Min: 1, Max: 1	Each vector entry represents exactly one text chunk.

The Vector Database Entry entity stores semantic representations used for retrieval.

- vectorId: String
- embeddingVector: Float[]

- chunkId: String

Similarity scores are calculated dynamically during query execution and are therefore not stored persistently.

Query Data $\langle D40 \rangle$

Relationship	Cardinality	Expected Data Volume	Description
produces Response	1:1	Min: 1, Max: 1	Each query generates exactly one response.

The Query entity stores user questions submitted to the system.

- queryId: String
- userInput: String
- timestamp: DateTime

Query embeddings are generated at runtime and are not stored persistently.

Response Data $\langle D50 \rangle$

Relationship	Cardinality	Expected Data Volume	Description
belongs to Query	1:1	Min: 1, Max: 1	Each response corresponds to exactly one query.
uses Text Chunks	n:m	Min: 1, Max: many	Responses are generated using one or more retrieved chunks as evidence.

The Response entity stores the final output generated by the system.

- responseId: String

- queryId: String
- generatedAnswer: String
- sourceChunks: List<String>

The source chunks provide traceability and allow users to verify the origin of generated answers through citations.

8 Configuration

This chapter describes the configuration of the PDF Search Agent and the parameters required for its deployment and operation. The system consists of a Streamlit-based web application, a document processing pipeline, a retrieval component, a vector database, a Large Language Model (LLM), and an evaluation pipeline. The configuration parameters presented in this chapter allow the system to be adapted to different deployment environments while maintaining reliable document retrieval and grounded answer generation.

8.1 Runtime Configuration

The PDF Search Agent is implemented in Python and provided as a web application using the Streamlit framework. The application can be executed on any operating system supporting Python, including Windows, Linux, and macOS.

Before starting the application, all required software dependencies must be installed. These dependencies are specified in the project's `requirements.txt` file located in the project root directory.

The most important dependencies are summarized in Table 8.1.

The required packages can be installed using

```
pip install -r requirements.txt
```

The application is started using the project's Streamlit entry point, for example

```
streamlit run ui/streamlit_app.py
```

Package	Purpose
Streamlit	Web-based graphical user interface
PyMuPDF	PDF parsing and text extraction
LangChain	Coordination of the Retrieval-Augmented Generation (RAG) pipeline
ChromaDB	Local vector database for semantic retrieval
Sentence Transformers	Embedding generation using the all-MiniLM-L6-v2 model

Table 8.1: Main runtime dependencies

The runtime environment should provide sufficient processing power and memory to execute PDF parsing, embedding generation, semantic retrieval, and answer generation efficiently.

8.2 Document Storage Configuration

Uploaded PDF documents are stored in a configurable storage directory within the project. The storage location can be adapted depending on the deployment environment.

For every uploaded document, the system stores the metadata shown in Table 8.2.

Metadata	Purpose
Document Identifier	Unique identifier of the uploaded document
File Name	Original document name displayed to the user
Upload Timestamp	Records when the document was uploaded
File Size	Used for validation and document management
Storage Path	Location of the stored PDF document

Table 8.2: Stored document metadata

These metadata support document management, indexing, semantic retrieval, and citation generation. The current implementation supports PDF documents only.

8.3 Retrieval Configuration

The retrieval component prepares uploaded documents for semantic search.

During document processing, the extracted document text is divided into document chunks. Each chunk is converted into an embedding vector using the Sentence Transformers `all-MiniLM-L6-v2` model and stored together with its metadata in the local ChromaDB vector database.

The retrieval behaviour can be configured using the parameters listed in Table 8.3.

Parameter	Description
Chunk Size	Maximum size of each document chunk
Chunk Overlap	Number of overlapping tokens between neighbouring chunks
Embedding Model	Sentence Transformers <code>all-MiniLM-L6-v2</code>
Top-k Retrieval	Number of retrieved document chunks returned to the language model
Cosine Similarity	Similarity function used during semantic retrieval
Retrieval Threshold	Minimum similarity score required before answer generation

Table 8.3: Retrieval configuration parameters

These parameters directly influence retrieval quality, indexing performance, memory consumption, and the amount of contextual information provided during answer generation.

8.4 Language Model Configuration

The answer generation component is based on a Large Language Model (LLM). The model receives the retrieved document chunks together with the user's query and generates an answer strictly based on the provided context.

The configurable parameters are summarized in Table 8.4.

Parameter	Description
Model	Selected Large Language Model
Temperature	Controls response randomness
Maximum Response Length	Maximum number of generated tokens
Context Window	Number of retrieved document chunks passed to the language model
Citation Settings	Enables document name and page number citations

Table 8.4: Language model configuration parameters

Lower temperature values produce more deterministic responses, whereas higher values increase response diversity. Citation settings ensure that generated answers include references to the source document and page number whenever supporting evidence is available.

8.5 User Interface Configuration

The graphical user interface is implemented using the Streamlit framework. Users interact with the application through a web browser to upload PDF documents, submit questions, and inspect generated answers together with their citations.

The configurable interface parameters are shown in Table 8.5.

Parameter	Description
Maximum Upload Size	Maximum permitted PDF file size
Displayed Search Results	Number of citations displayed together with an answer
Citation Visibility	Enables or disables citation display
Sidebar Layout	Configuration of the document management area
Administration Dashboard	Enables access to evaluation and administration functions

Table 8.5: User interface configuration parameters

The Streamlit framework hosts the application locally and provides browser-based access without requiring additional software on the client side.

8.6 Evaluation Configuration

The PDF Search Agent includes an evaluation pipeline that measures the quality of the Retrieval-Augmented Generation system using a standardized benchmark.

The evaluation pipeline processes a selected subset of the Open RAG Benchmark and computes quality metrics using the RAGAS evaluation framework together with additional project-specific metrics.

The configurable evaluation parameters are listed in Table 8.6.

Parameter	Description
Benchmark Dataset	Selected subset of the Open RAG Benchmark
Evaluation Metrics	Faithfulness, Answer Relevancy, Context Recall, Context Precision
Additional Metrics	Citation Correctness, Abstention Rate, Average Latency
Output Directory	Directory used to store evaluation results
Output Format	JSON and CSV result files

Table 8.6: Evaluation configuration parameters

Evaluation results are stored in the `evaluation/results/` directory as `scores.json` and `scores.csv`. These files enable reproducible experiments and support the comparison of different system configurations with respect to retrieval quality, answer quality, citation correctness, abstention behaviour, and system efficiency.

8.7 Summary

The configuration of the PDF Search Agent provides flexible deployment while maintaining reliable document processing and grounded question answering. Configurable parameters allow the

runtime environment, document storage, retrieval component, language model, user interface, and evaluation pipeline to be adapted to different deployment scenarios without modifying the overall system architecture.

9 Changes Compared to the Functional Design

This chapter describes the refinements introduced during the transition from the Functional Design to the Technical Design. The overall functionality of the PDF Search Agent remains unchanged. The system still follows the same Retrieval-Augmented Generation (RAG) workflow consisting of PDF ingestion, text extraction, chunking, embedding generation, retrieval, answer generation, abstention handling, and evaluation.

The Technical Design builds upon the concepts presented in the Functional Design by providing implementation-oriented specifications, software module descriptions, configurable parameters, and technical decisions required for the implementation of the system.

9.1 Architecture Refinement

The Functional Design describes the logical architecture of the system and the interaction between its major components. The Technical Design refines this architecture by providing a more detailed description of the individual software modules and their responsibilities.

The implementation modules, including the parser, chunker, indexer, retriever, generator, and pipeline controller, are described with clearly defined interfaces and responsibilities. This additional level of detail improves modularity, maintainability, and testability while preserving the overall system architecture defined in the Functional Design.

The Technical Design also specifies how data flows between the individual modules, allowing each component to be implemented, tested, and modified independently.

9.2 PDF Processing Pipeline

The overall PDF processing workflow remains unchanged. The Technical Design provides a more detailed description of how each processing step is carried out during implementation.

The PDF validation process specifies technical checks such as verifying the PDF file format, ensuring that the uploaded file is not empty, and confirming that readable text can be extracted successfully.

The text extraction process is further specified by describing page-by-page processing. Each extracted page is associated with its corresponding page number, allowing retrieved information to reference the exact location within the original document.

The Technical Design also makes the chunking configuration explicit by introducing configurable chunk size and chunk overlap parameters. These parameters allow the retrieval behaviour to be adapted without modifying the overall processing pipeline.

9.3 Retrieval Pipeline

The retrieval workflow follows the same process defined in the Functional Design. The Technical Design primarily refines the implementation details and configuration of the retrieval component.

Document chunks are converted into embedding vectors using the Sentence Transformers `all-MiniLM-L6-v2` model and stored in the ChromaDB vector database together with their metadata. User queries are embedded using the same model to ensure semantic consistency between document representations and search queries.

The retrieval component additionally exposes configurable parameters, including chunk size, chunk overlap, the number of retrieved chunks, cosine similarity search, and retrieval thresholds. These parameters support systematic evaluation and optimization of retrieval quality while maintaining the retrieval workflow defined in the Functional Design.

9.4 Answer Generation

The Functional Design specifies that answers are generated exclusively from retrieved document content. The Technical Design extends this description by providing implementation details for prompt construction and citation generation.

Before the language model is invoked, the retrieved document chunks are combined with the user's question to construct the prompt supplied to the model. After answer generation, duplicate citations are removed and the final response is returned together with the corresponding document names and page numbers.

These refinements improve readability, traceability, and maintainability without changing the functional behaviour of the system.

9.5 Abstention Mechanism

The abstention mechanism remains conceptually unchanged but is specified in greater technical detail.

The Functional Design defines that the system should abstain whenever insufficient evidence is available. The Technical Design specifies the decision process used to implement this behaviour.

After retrieval, the system evaluates the similarity scores of the retrieved document chunks. If the similarity score is below the configured threshold, the system immediately returns a predefined abstention message without generating an answer.

A second verification step is performed after answer generation. If the language model indicates that no reliable answer can be produced from the provided context, the generated response is replaced by the predefined abstention message.

These implementation details reduce the probability of unsupported or hallucinated responses while preserving the behaviour defined in the Functional Design.

9.6 Data Model

The overall structure of the persistent data model remains unchanged. The entities Document Data, Text Chunk Data, Vector Database Entries, Query Data, and Response Data continue to represent the core data objects of the system.

The Technical Design extends the description of these entities by specifying their role during implementation. Metadata such as document name, page number, chunk index, and token count are described in greater detail because they are required for semantic retrieval, citation generation, and evaluation.

In addition, the Technical Design clarifies that similarity scores are computed dynamically during query execution rather than being stored persistently. This reduces storage requirements while maintaining retrieval efficiency.

9.7 Configuration

The Functional Design defines the major configuration categories of the system. The Technical Design expands these categories by specifying the individual configuration parameters required during implementation.

In addition to the runtime environment, document storage, retrieval configuration, language model configuration, and user interface settings, the Technical Design introduces a dedicated Evaluation Configuration section.

The configurable parameters include chunk size, chunk overlap, embedding model selection, cosine similarity, retrieval thresholds, the number of retrieved chunks, language model parameters, citation settings, benchmark configuration, evaluation metrics, and result storage locations.

Making these parameters explicit enables reproducible experiments and allows different system configurations to be compared systematically during testing and evaluation.

9.8 Evaluation

The overall evaluation concept remains unchanged. The Technical Design provides additional implementation details describing how evaluation is performed automatically.

The complete Retrieval-Augmented Generation pipeline is executed using benchmark questions from the Open RAG Benchmark dataset. The Technical Design specifies how benchmark data are processed, how prediction results are stored, and how evaluation metrics are calculated.

In addition to the RAGAS metrics Faithfulness, Answer Relevancy, Context Recall, and Context Precision, the system computes Citation Correctness, Abstention Rate, and Average Latency. The evaluation results are exported as structured JSON and CSV files, enabling reproducible experiments and direct comparison between different system configurations.

9.9 Summary

The Technical Design does not introduce major functional changes to the PDF Search Agent. Instead, it refines the concepts defined in the Functional Design by providing implementation-oriented specifications, software module descriptions, configurable system parameters, and detailed evaluation procedures.

These refinements establish a clear transition from the conceptual system design to an implementable software architecture while preserving the original objectives and behaviour of the PDF Search Agent.

10 Fulfilment of Requirements

This chapter describes how the requirements defined in the Requirements Specification are fulfilled by the implemented PDF Search Agent. For each requirement, the corresponding product functions and software components responsible for its implementation are identified.

Table 10.1 provides an overview of the traceability between the requirements, the functional design, and the implementation.

Requirement	Product Functions	Main Components
RM1	F10	Streamlit UI, PDF Parser, Indexer
RM2	F20–F50	Parser, Chunker, Indexer, Retriever, Generator
RM3	F40–F60	Retriever, Generator
RM4	F10, F60	Parser, Streamlit UI, Generator
RM5	F10–F60	Complete modular architecture
RM6	Evaluation Pipeline	Benchmark Loader, RAG Pipeline, RAGAS

Table 10.1: Requirement traceability

10.1 Mandatory Requirements

The following mandatory requirements are fulfilled by the implemented system.

RM1: Import and Management of PDF Documents

RM1 is fulfilled by the Streamlit user interface together with the PDF Parser and Indexer modules.

The user can upload one or multiple PDF documents through the graphical user interface. Every uploaded document is validated before processing. Successfully validated documents are parsed, indexed, and stored together with their metadata so that they become available for semantic retrieval and citation generation.

RM2: Extraction and Question Answering

RM2 is implemented jointly by the Parser, Chunker, Indexer, Retriever, and Generator modules.

The parser extracts the document text, the chunker divides the text into document chunks, the indexer generates embeddings and stores them in the vector database, the retriever identifies the most relevant document chunks, and the generator produces grounded answers together with the corresponding document and page citations.

RM3: Abstention

RM3 is fulfilled by the Retrieval and Answer Generation components.

After retrieval, the similarity scores of the retrieved document chunks are evaluated. If insufficient evidence is available, the system returns a predefined abstention message instead of generating an unsupported answer. A second verification step after answer generation further reduces the probability of hallucinated responses.

RM4: Error Handling

RM4 is implemented throughout the complete processing pipeline.

The system validates uploaded PDF documents, detects unsupported or corrupted files, handles empty user queries, reports missing indexed documents, and catches processing errors during

parsing, retrieval, and answer generation. In all cases, informative error messages are presented to the user while the application remains operational.

RM5: Reliability and Robustness

RM5 is fulfilled by the modular software architecture of the PDF Search Agent.

The separation into independent components for parsing, chunking, indexing, retrieval, answer generation, and evaluation improves maintainability and allows individual modules to be tested independently. This modular design also simplifies future extensions and supports systematic evaluation.

RM6: Evaluation Pipeline

RM6 is fulfilled by the evaluation pipeline.

The evaluation pipeline executes benchmark questions using the complete Retrieval-Augmented Generation pipeline and computes quantitative quality metrics. The implementation uses the Open RAG Benchmark together with the RAGAS evaluation framework and additionally measures Citation Correctness, Abstention Rate, and Average Latency. The resulting scores are stored in structured JSON and CSV files for later inspection.

10.2 Desired Requirements

The following desired requirements are fulfilled by the implemented system.

RS1: Semantic Search

RS1 is fulfilled by the semantic retrieval component.

Instead of relying solely on keyword matching, the system converts both document chunks and user queries into embedding vectors and retrieves the most semantically similar document passages from the ChromaDB vector database.

RS2: Structured Answers

RS2 is fulfilled by the Answer Generation module.

Generated answers are presented in a structured format together with the corresponding document names and page numbers, allowing users to verify the source of the generated information.

RS3: User-Friendly Interface

RS3 is fulfilled by the Streamlit-based graphical user interface.

The interface enables users to upload PDF documents, submit natural language questions, inspect generated answers, and review citations through a simple browser-based application.

RS4: Incremental Updates

RS4 is fulfilled by the indexing component.

Additional PDF documents can be processed individually without requiring the complete document collection to be reprocessed, allowing the indexed corpus to be extended incrementally.

10.3 Optional Requirements

The following optional requirements are not part of the current system implementation.

RC1: Enhanced Graphical User Interface

The current Streamlit interface provides all required functionality. Additional graphical improvements are outside the scope of the current implementation.

RC2: Export of Answers

This requirement is not implemented in the current version of the system.

RC3: Multilingual Support

This requirement is not implemented in the current version of the system.

RC4: Query History

This requirement is not implemented in the current version of the system.

RC5: Document Summaries

This requirement is not implemented in the current version of the system.

11 Glossary

Table 11.1: Glossary of technical terms

Term	Description
Abstention	Mechanism that prevents the system from generating an answer when the retrieved evidence is insufficient.
Chunk	A smaller text segment created from a PDF document during preprocessing to improve semantic retrieval.
ChromaDB	Local vector database used to store embedding vectors and perform semantic similarity search.
Citation	Reference to the original source document and page number supporting a generated answer.
Cosine Similarity	Similarity measure used to compare embedding vectors during semantic retrieval.
Embedding	Numerical vector representation of text that captures its semantic meaning.
Indexing	Process of generating embeddings for document chunks and storing them in the vector database.
LangChain	Framework used to coordinate the Retrieval-Augmented Generation (RAG) pipeline.
Large Language Model (LLM)	Artificial intelligence model used to generate natural-language answers from retrieved document context.

Term	Description
Ollama	Local framework used to execute the Llama language model without relying on external cloud services.
PDF Parser	Component responsible for validating uploaded PDF documents and extracting their text content.
PyMuPDF	Python library used for PDF parsing and text extraction.
RAG (Retrieval-Augmented Generation)	Question-answering approach that combines semantic document retrieval with a language model to generate grounded answers.
RAGAS	Evaluation framework used to assess the quality of Retrieval-Augmented Generation systems using metrics such as Faithfulness and Answer Relevancy.
Retriever	Component that searches the vector database and returns the document chunks most relevant to a user's question.
Sentence Transformers	Neural embedding models used to convert documents and queries into semantic vector representations.
Streamlit	Python framework used to implement the graphical web-based user interface.
Top-k Retrieval	Configuration parameter defining how many of the most similar document chunks are returned during semantic search.
Vector Database	Database optimized for storing embedding vectors and performing similarity search.